# ÇANKAYA UNIVERSITY
# COMPUTER ENGINEERING DEPARTMENT

## CENG 408

# System Architecture
## and
# Methodology

## TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

# TABLE OF CONTENTS


**1. Introduction....3**
1.1. Purpose....3
1.2. Definitions and Acronyms....3
**2. System Overview....7**
**3. System Architecture....7**
3.1. Architectural Design....7
3.1.1. Layered Architecture....7
3.1.1.1. Presentation Layer....7
3.1.1.2. Application & Analytics Layer....7
3.1.1.3. Data & Cloud Services Layer....8
3.1.1.4. Device & Sensing Layer....8
3.1.1.5. Communication Layer....8
3.1.2. Client–Server Architecture....8
3.2. Decomposition Description....8
3.2.1. Field Sensing & Data Acquisition Subsystem....8
3.2.2. Analytics & Decision Support Subsystem....8
3.2.3. Visualization & User Interaction Subsystem....9
3.2.4. Class Diagram....9
3.3. System Modeling....10
3.3.1. Activity Diagrams....10
3.3.2. Sequence Diagrams....27
**4. Methodology....38**
4.1. Irrigation Prediction....38
4.1.1. Problem Formulation....38
4.1.2. Data Acquisition and Preprocessing....39
4.1.3. Algorithmic Approach....39
4.1.4. System Evaluation Metrics....40
4.1.5. Implementation Details....41
4.2. Disease Detection....41
4.2.1. Problem Formulation....41
4.2.2. Data Acquisition and Preprocessing....42
4.2.3. Algorithmic Approach....42
4.2.4. Model Selection and Comparative Evaluation....43
4.2.5. System Evaluation Metrics....43
4.2.6. Implementation Details....43
4.3. LLM-Based Advisory & Decision Support....44
4.3.1. Problem Formulation....44
4.3.2. Input Representation and Context Construction....44
4.3.3. Response Generation and Explainability....44
4.3.4. System Evaluation and Limitations....45
4.3.5. Implementation Details....45
4.4. Sustainability & Carbon Footprint Calculation....46
4.4.1. Problem Formulation....46
4.4.2. Data Acquisition & Input Representation....46
4.4.3. Calculation Approach....47
4.4.4. Output Metrics & Interpretation....47
4.4.5. Implementation Details....48
**5. Data Design....48**
5.1. Entity-Relationship Diagram....48
5.2. Data Pipeline....49
**6. User Interface Design....50**
6.1. Interface Hierarchy....50
6.2. Visual Presentation....50
6.3. Accessibility and Responsivity....51
**7. Conclusion....58**
**8. References....59**

# 1. Introduction

## 1.1. Purpose

This document defines the system architecture of the TARAS precision agriculture platform. It presents the high-level structural design of the system by describing its core architectural components, logical layers, and their interactions within the overall platform. The architectural decisions described in this document are derived directly from the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

In addition to the architectural design, the document outlines the methodological approach adopted for data acquisition, preprocessing, and irrigation decision logic, explaining how agronomic principles and sensor-driven analytics are translated into operational system behavior. It also describes the role of auxiliary AI-assisted advisory components, which support explainability and user understanding without performing core decision-making.

The document serves as an architectural and methodological reference for guiding system implementation, integration, and future evolution, ensuring a consistent and maintainable realization of the TARAS platform.

## 1.2. Definitions and Acronyms

| **Term** | **Definition** |
|---|---|
| Accuracy | The ratio of correctly classified instances (both positive and negative) to the total number of instances. It represents the overall correctness of the model's predictions. |
| Precision | The proportion of correctly predicted positive instances among all instances predicted as positive. It indicates how reliable the positive predictions of the model are. |
| Recall | The proportion of correctly predicted positive instances among all actual positive instances. It measures the model's ability to identify all relevant positive cases. |
| F1-Score | The harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives, especially useful when class distributions are imbalanced. |
| Agronomy | The branch of agricultural science that focuses on crop production and soil management, examining the interactions between crops, soil, climate, and agricultural practices in order to optimize yield and resource efficiency. |
| JWT(JSON Web Token) | JWT is a compact and secure token format used for authentication and authorization, allowing the system to verify user identity and access rights between the client and backend services. |

| | |
|---|---|
| ML | ML refers to computational techniques that enable systems to learn patterns from data and make predictions or decisions without being explicitly programmed, and is used in the TARAS system primarily for plant disease detection and auxiliary analysis tasks. |
| CV Models | CV models are computational models designed to analyze and interpret visual data such as images, enabling tasks like object recognition, classification, and pattern detection. In the TARAS system, they are used for plant leaf disease detection. |
| Cloud Services | Cloud services refer to on-demand computing resources such as storage, databases, and backend services provided over the internet, used in the TARAS system to support data storage, processing, and system scalability. |
| Client | The client is the user-facing component of a client–server architecture that initiates requests to the server and presents system functionality and data to the user. In the TARAS system, the mobile application acts as the client. |
| Server | The server is the backend component of a client–server architecture that receives requests from clients, processes them, manages data storage and system logic, and returns responses. In the TARAS system, the server hosts backend services, databases, and analysis modules. |
| Storage Bucket | A storage bucket is a logical container in cloud storage systems used to store and organize unstructured data such as images and files. In the TARAS system, storage buckets are used to store uploaded plant images and related assets. |
| Classification | Classification is a ML task in which input data are assigned to one of several predefined categories or classes. In the TARAS system, classification is used to identify plant diseases from leaf images. |
| Processing Pipeline | The process of collecting raw data from heterogeneous sources and transforming it into a consistent, reliable, and analysis-ready format. This includes acquiring environmental sensor data, historical records, metadata, and image data, followed by preprocessing steps such as timestamp alignment, unit normalization, quality control, outlier detection, and missing data handling to ensure data integrity before model-based analysis. |
| Deep Learning | A subfield of ML that utilizes neural networks with multiple layers to automatically learn hierarchical feature representations from data. It is particularly effective for complex tasks such as image recognition, natural language processing, and speech analysis. |
| CNN | A deep learning architecture designed to automatically extract features from image data using convolutional layers. It is widely used in tasks such as image classification and object recognition. |
| Overfitting | Overfitting refers to a modeling phenomenon where a ML model learns training data too closely, resulting in high training performance but poor generalization to unseen data. |

| | |
|---|---|
| Pre-trained weights | Model parameters obtained by training a neural network on a large-scale dataset prior to being applied to a target task. They are used to accelerate training and improve generalization performance. |
| Finetune | The process of retraining some or all layers of a pre-trained model on a task-specific dataset using a low learning rate, allowing the model to adapt to the new problem domain. |
| TensorFlow | An open-source ML framework developed by Google for building, training, and deploying ML and deep learning models. |
| Keras | A high-level deep learning API that runs on top of TensorFlow, enabling fast and efficient development of neural network models with a user-friendly and modular structure. |
| WCAG (Web Content Accessibility Guidelines) | WCAG is a set of international guidelines that define standards for making digital interfaces accessible to users with diverse abilities, focusing on principles such as perceivability, operability, understandability, and robustness. |
| Cognitive Load | Cognitive load refers to the amount of mental effort required to process information and interact with a system. In the TARAS interface design, reducing cognitive load aims to make the application easier to understand and use, especially for users with limited technical experience. |
| Carbon Footprint | The total amount of greenhouse gas emissions (expressed as $CO_2$-equivalent) produced directly and indirectly by agricultural activities. |
| $CO_2$ equivalent ($CO_2e$) | A standardized unit used to express the impact of different greenhouse gases in terms of the amount of $CO_2$ that would produce the same warming effect. |
| Emission Factor | A coefficient that converts activity data (such as fuel consumption or electricity use) into greenhouse gas emissions. |
| Greenhouse Gas (GHG) | Gases that trap heat in the atmosphere, including $CO_2$, $CH_4$ (methane), and $N_2O$ (nitrous oxide), contributing to global warming. |
| Activity Data | Quantitative data representing human activities that generate emissions, such as fuel usage, electricity consumption, or fertilizer application. |
| Sustainability | The practice of managing resources in a way that meets current needs without compromising future environmental, economic, and social systems. |
| Environmental Impact | The effect of agricultural activities on the environment, including emissions, water usage, and resource consumption. |
| Normalized Emissions | Emission values adjusted relative to a reference unit, such as per hectare or per unit of production, to enable comparison. |
| Tier 1 (IPCC | A simplified estimation method defined by IPCC that uses default |

| | |
|---|---|
| Methodology) | emission factors and basic activity data for calculating greenhouse gas emissions. |
| IPCC | Intergovernmental Panel on Climate Change, an international body providing guidelines for greenhouse gas estimation. |
| FAO | Food and Agriculture Organization of the United Nations, providing datasets and methodologies for agriculture-related analysis. |

# 2. System Overview

The TARAS (Tarım Analizi Sistemi) platform is a precision agriculture decision-support system designed to assist farmers in irrigation planning, crop health monitoring and sustainability assessment. The project initially explored a wide range of smart agriculture features, which were evaluated based on impact, feasibility, usability, cost and sustainability. Based on literature analysis and comparisons with existing solutions, the system scope was refined to focus on the most effective and implementable components. The final design centers on integrated field sensing, AI-driven analysis, digital twin visualization and a mobile application interface, ensuring a practical, scalable and user-oriented solution.

# 3. System Architecture

## 3.1. Architectural Design

The architectural design of the TARAS follows a hybrid architecture that combines Layered Architecture and Client–Server Architecture principles. This approach supports modular development, scalability and clear separation of responsibilities across system components.

### 3.1.1. Layered Architecture

#### 3.1.1.1. Presentation Layer

Provides the mobile application interface through which farmers interact with the system. This layer presents sensor readings, irrigation recommendations, disease detection results, digital twin visualizations, sustainability metrics such as carbon footprint, and AI-assisted advisory explanations in an accessible and user-friendly manner. The advisory interface translates analytical outputs into clear, context-aware guidance, supporting user understanding without replacing farmer decision-making.

#### 3.1.1.2. Application & Analytics Layer

Implements the core system logic, including data processing, irrigation prediction, disease analysis, decision-support workflows and user interaction management. ML and computer vision models operate within this layer to transform raw data into actionable insights.

#### 3.1.1.3. Data & Cloud Services Layer

Handles data storage, retrieval and synchronization between field devices and the mobile application. This layer manages time-series sensor data, historical records, model outputs and user inputs using a centralized cloud-based backend.

#### 3.1.1.4. Device & Sensing Layer

Includes IoT sensor nodes deployed in the field, responsible for collecting soil moisture, temperature and environmental data. This layer abstracts hardware-level interactions and ensures reliable data acquisition.

#### 3.1.1.5. Communication Layer

Manages data transmission between sensor nodes, gateway devices and cloud services using long-range and internet-based communication protocols. It ensures reliable, low-power and fault-tolerant data flow across the system.

### 3.1.2. Client–Server Architecture

The TARAS mobile application operates as a client that interacts with the cloud-based backend services. Field sensor data, user inputs, and system outputs, such as irrigation recommendations, disease analysis results and sustainability metrics are exchanged through request–response communication mechanisms. This separation between client and server components enables loose coupling, supports scalability and simplifies maintenance and future extensions.

## 3.2. Decomposition Description

### 3.2.1. Field Sensing & Data Acquisition Subsystem

- Soil Moisture Sensing Module
- Environmental Data Collection (Temperature, Humidity)
- Sensor Calibration & Validation Module
- Gateway Communication Module

### 3.2.2. Analytics & Decision Support Subsystem

- Data Storage Module
- Irrigation Prediction Module
- Disease Detection Module
- Sustainability & Carbon Footprint Calculation Module
- Data Processing & Model Management Module
- LLM advisory & Explainability Module

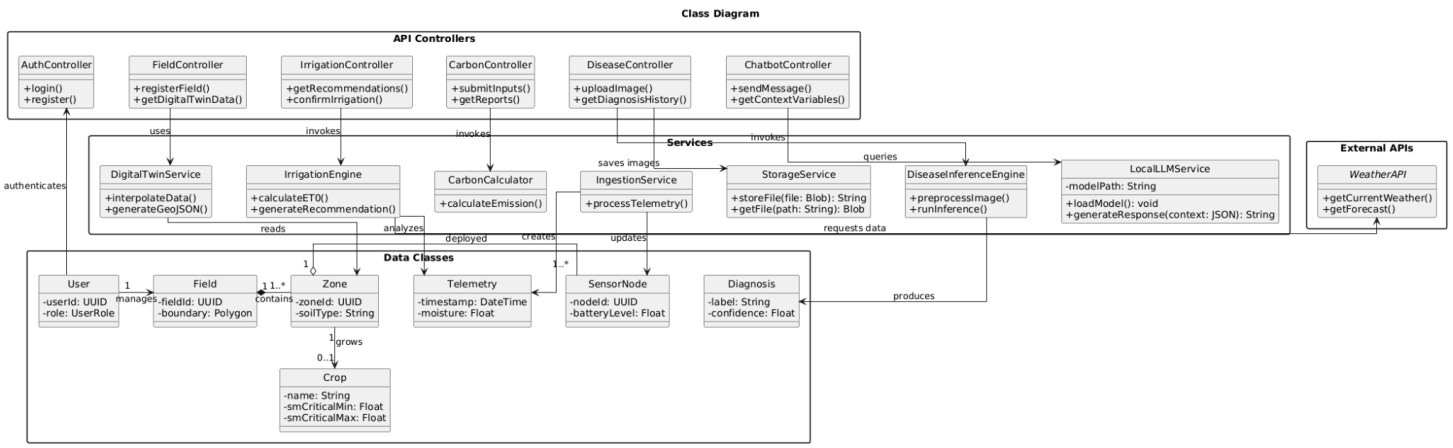
### 3.2.3. Visualization & User Interaction Subsystem

- Mobile Application Interface
- Digital Twin Visualization Module
- Recommendation & Alert Presentation
- User Input & Configuration Management

Each module in the TARAS platform interacts with other components through clearly defined interfaces, encapsulating internal logic and exposing only essential operations such as retrieving sensor measurements, generating irrigation recommendations, performing disease analysis, or updating visualization data. This approach supports abstraction, modular refinement, and reusability in line with core software engineering design principles.

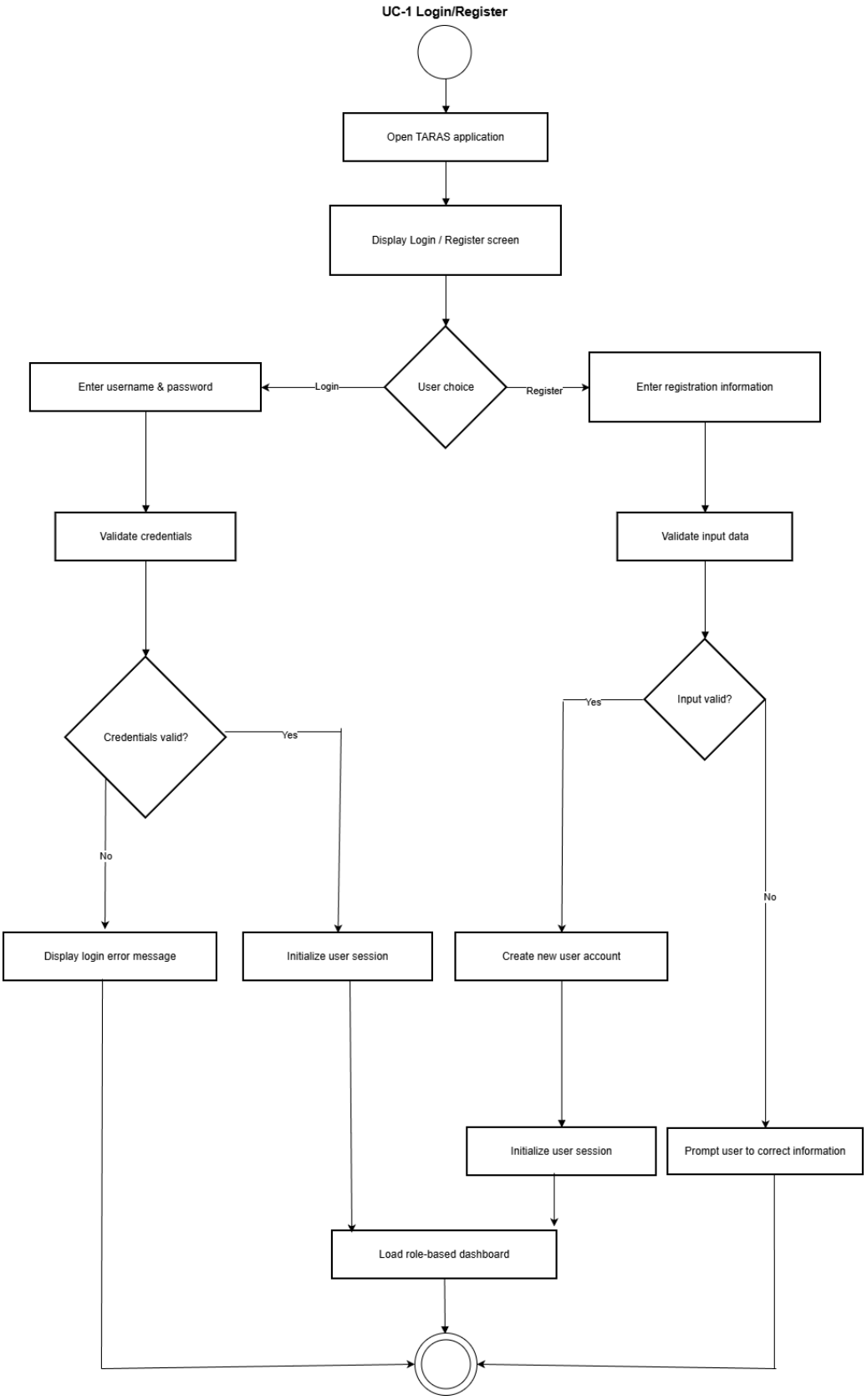
The modular system structure simplifies independent development and testing of components and allows future extensions, such as integrating new sensor types, improving prediction models or expanding sustainability metrics without impacting the overall system architecture.

### 3.2.4. Class Diagram



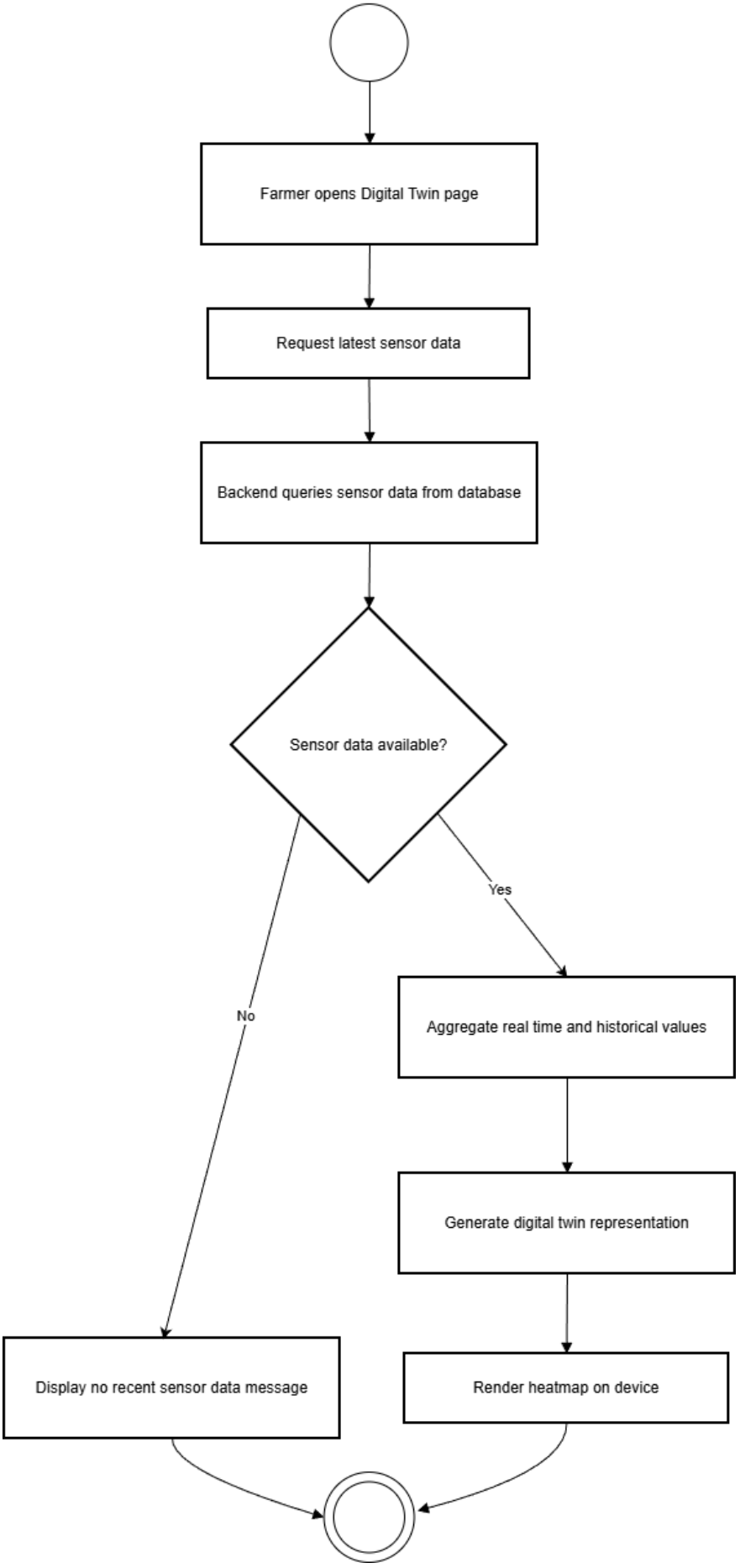
## 3.3. System Modeling

### 3.3.1. Activity Diagrams

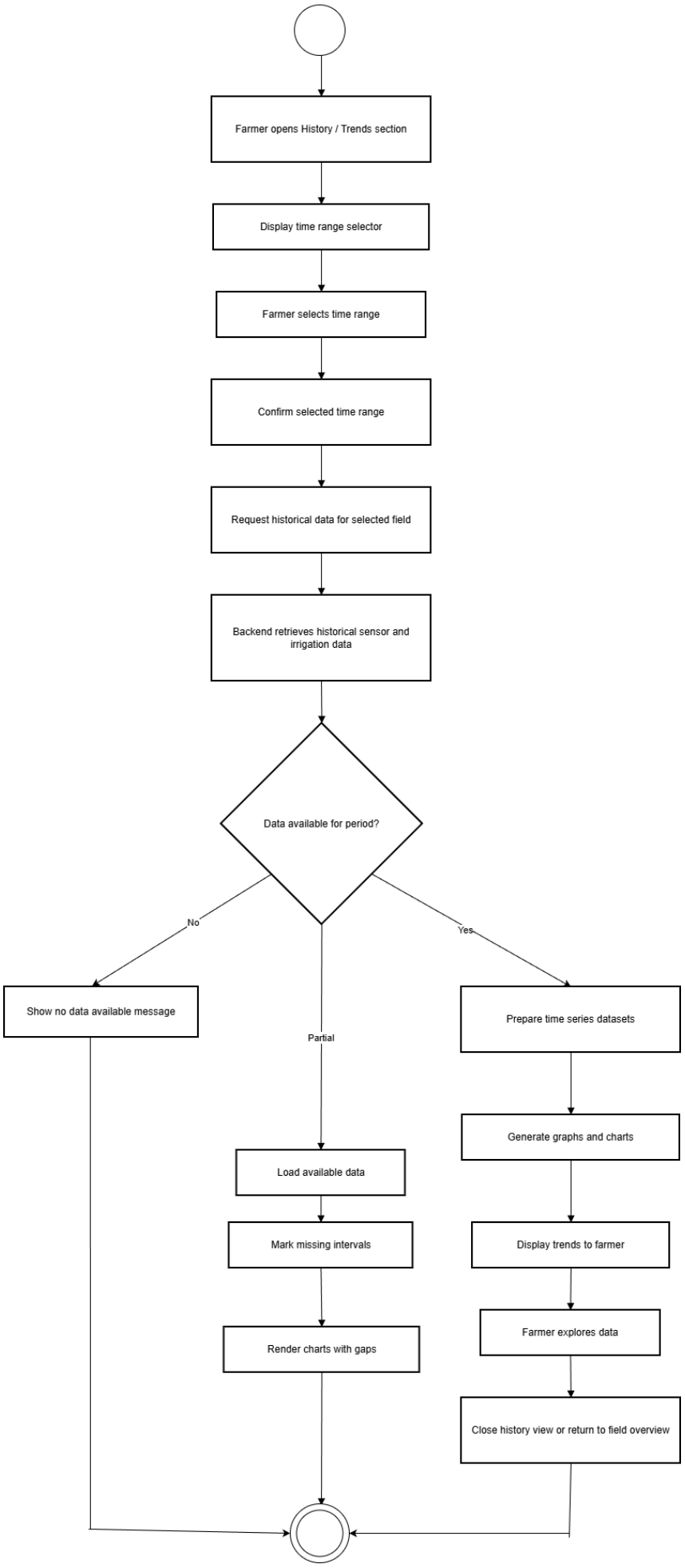


UC-2 New Field Registration
Farmer logged in
Open New Field Registration page
Scan for nearby sensor nodes
or retrieve available node IDs
Sensor nodes detected?
No
Yes
Notify farmer: No sensor nodes found
Display available sensor node IDs
Farmer selects sensor nodes
Validate selected node IDs
IDs valid and unique?
No
Yes
Request correction from farmer
Create new field record
Link selected sensor nodes to field
Confirm field registration

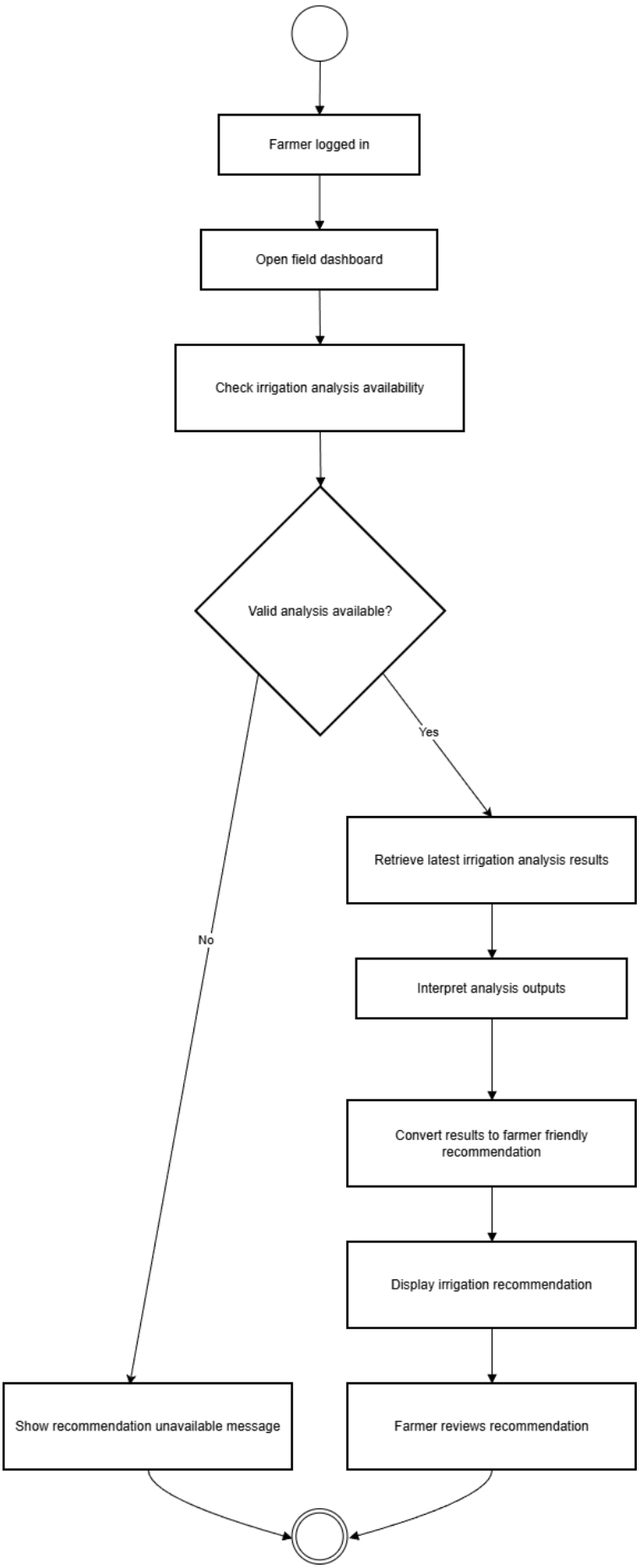
**UC-3 View Digital Twin**

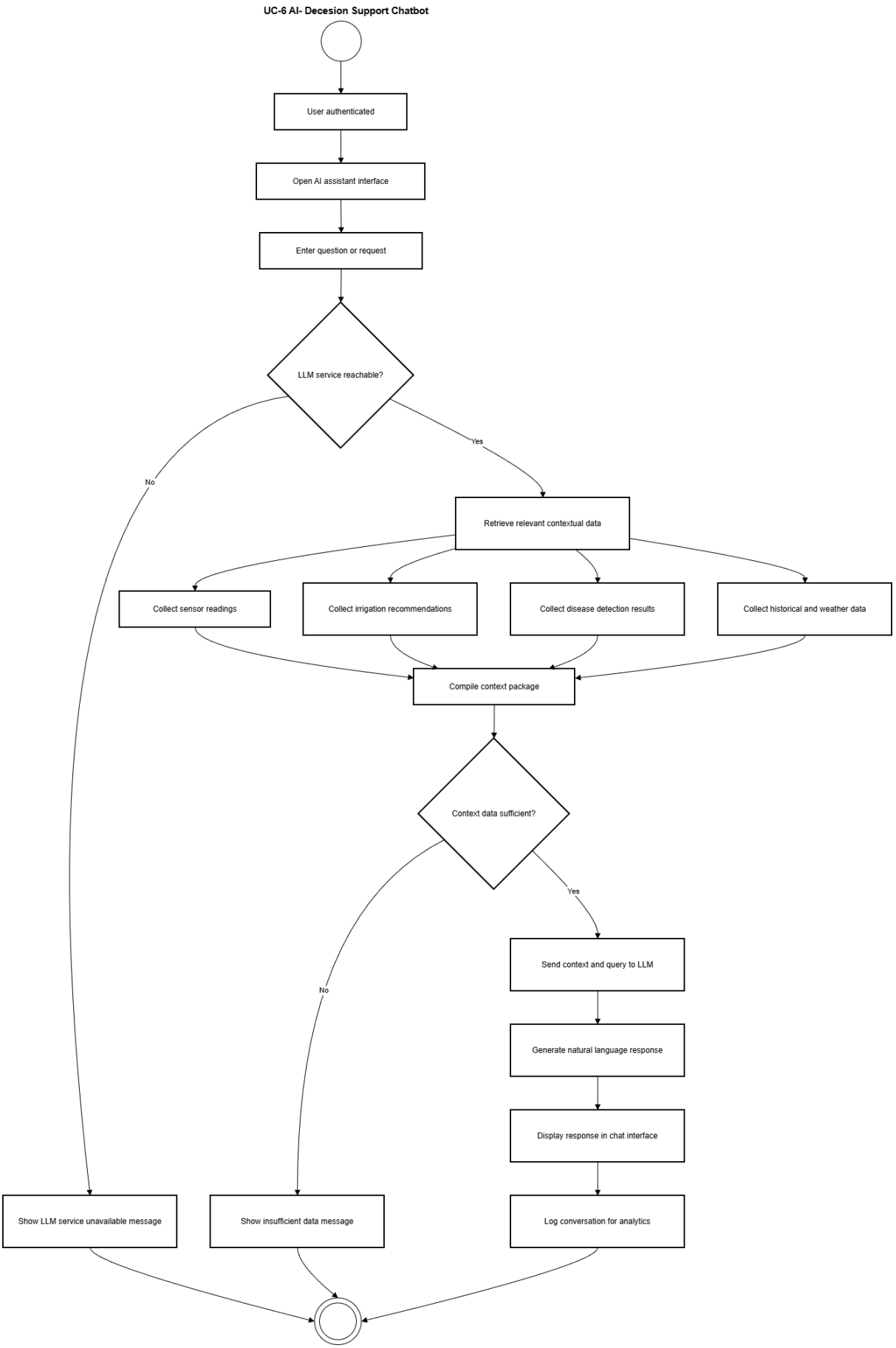


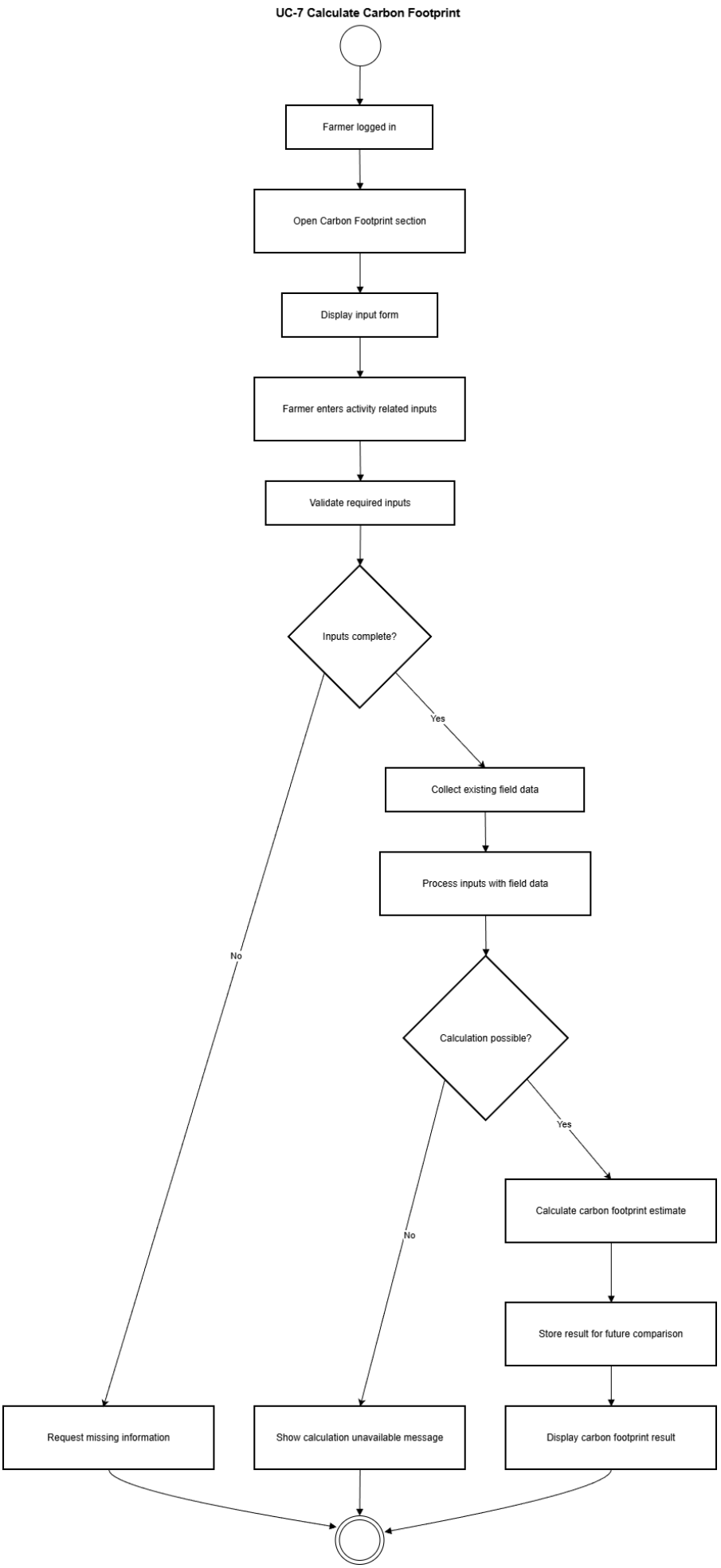
**UC-4 View Historical Data/Trends**



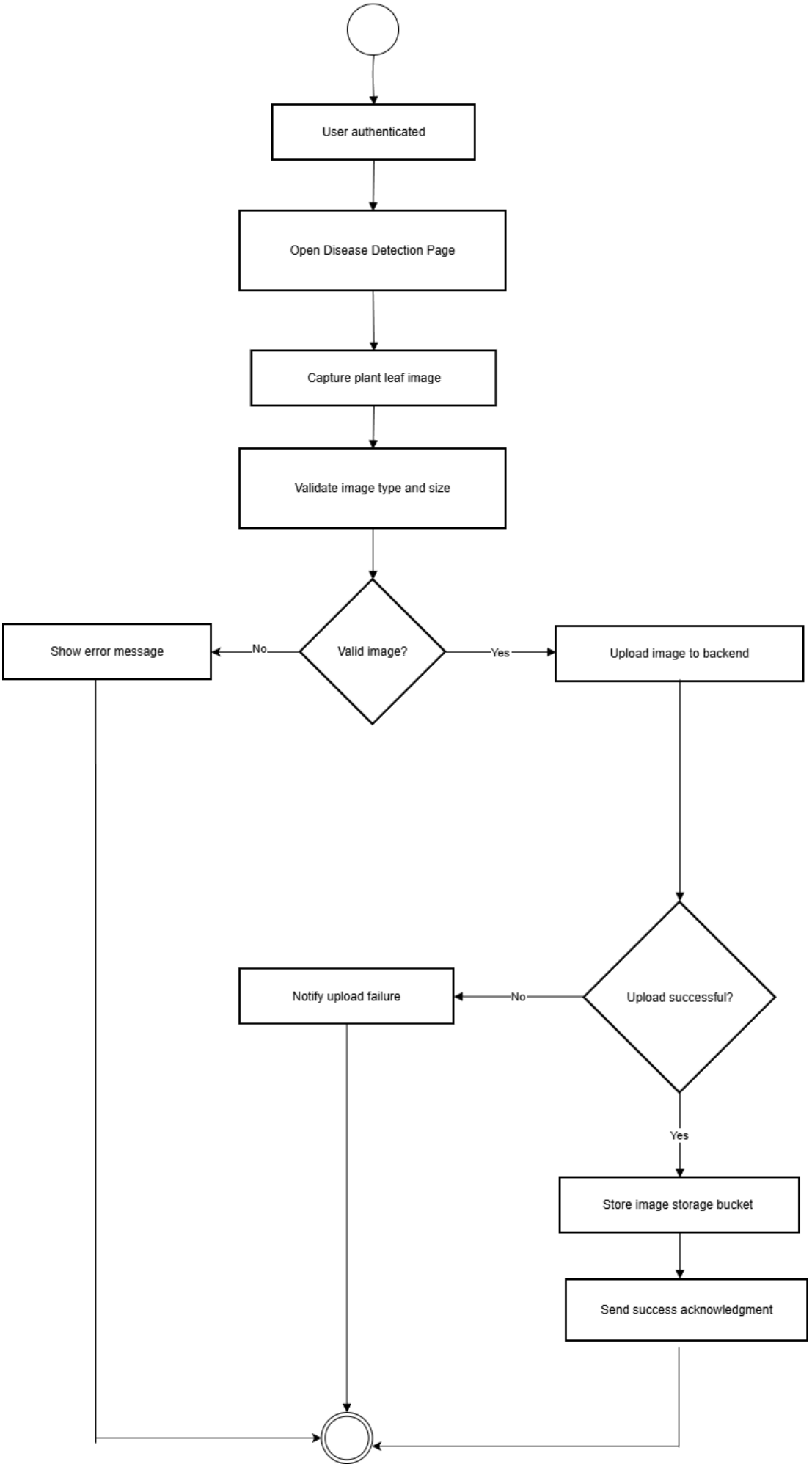
**UC-5 Receive Irrigation Suggestion**



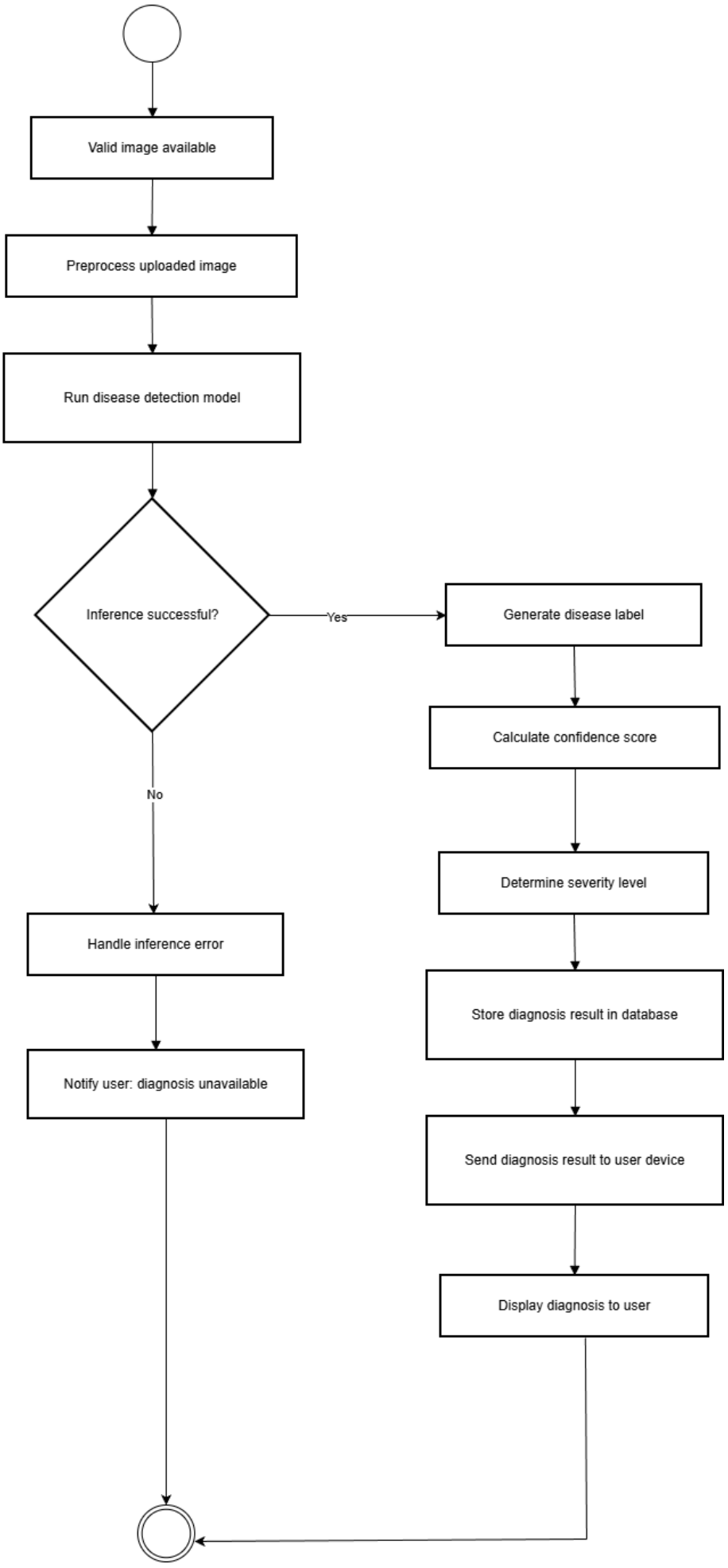

UC-6 AI- Decesion Support Chatbot
User authenticated
Open AI assistant interface
Enter question or request
LLM service reachable?
Yes
No
Retrieve relevant contextual data
Collect sensor readings
Collect irrigation recommendations
Collect disease detection results
Collect historical and weather data
Compile context package
Context data sufficient?
Yes
No
Send context and query to LLM
Generate natural language response
Display response in chat interface
Log conversation for analytics
Show LLM service unavailable message
Show insufficient data message


UC-7 Calculate Carbon Footprint
Farmer logged in
Open Carbon Footprint section
Display input form
Farmer enters activity related inputs
Validate required inputs
Inputs complete?
Yes
No
Collect existing field data
Process inputs with field data
Calculation possible?
Yes
No
Calculate carbon footprint estimate
Store result for future comparison
Request missing information
Show calculation unavailable message
Display carbon footprint result

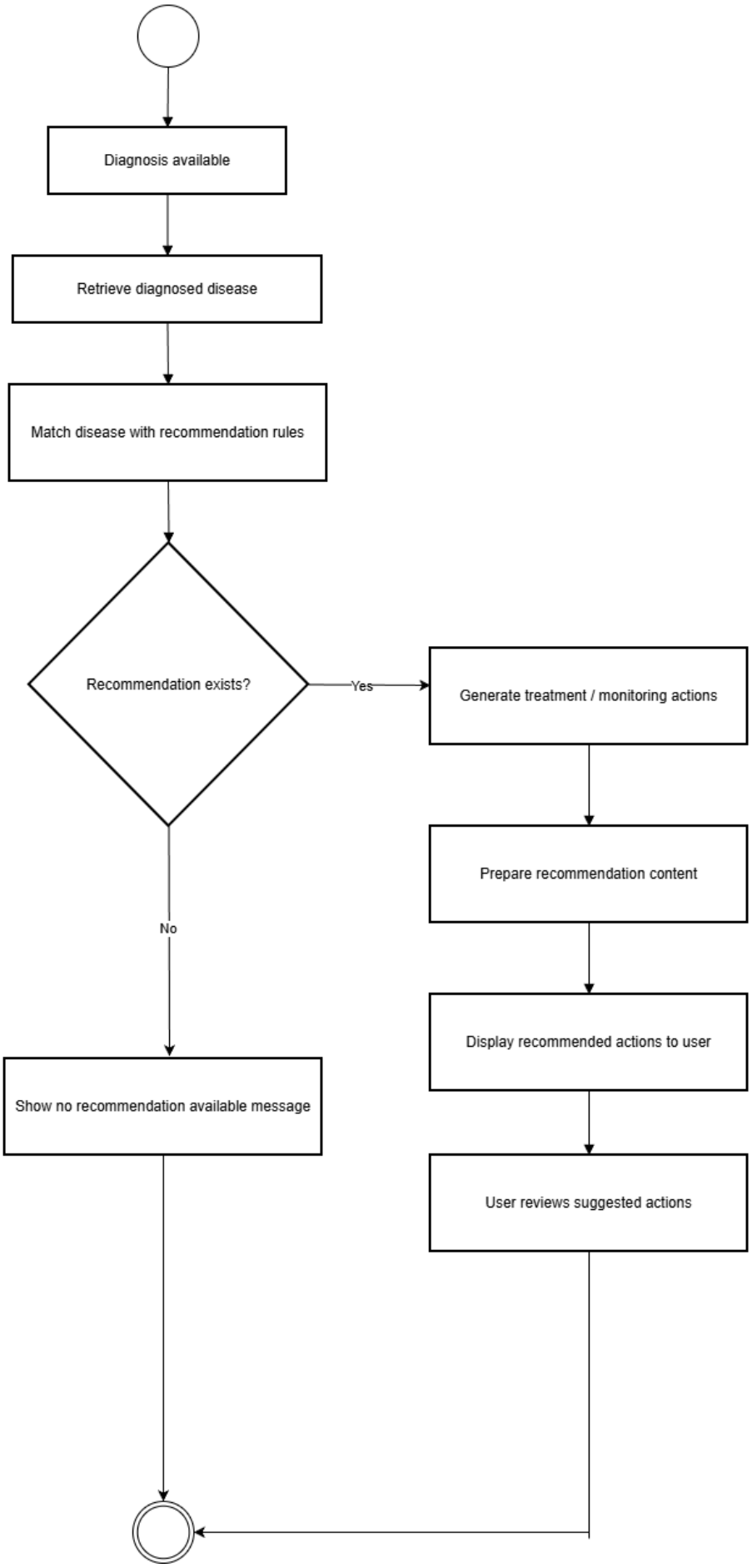
**UC-8 Upload Plant Image**

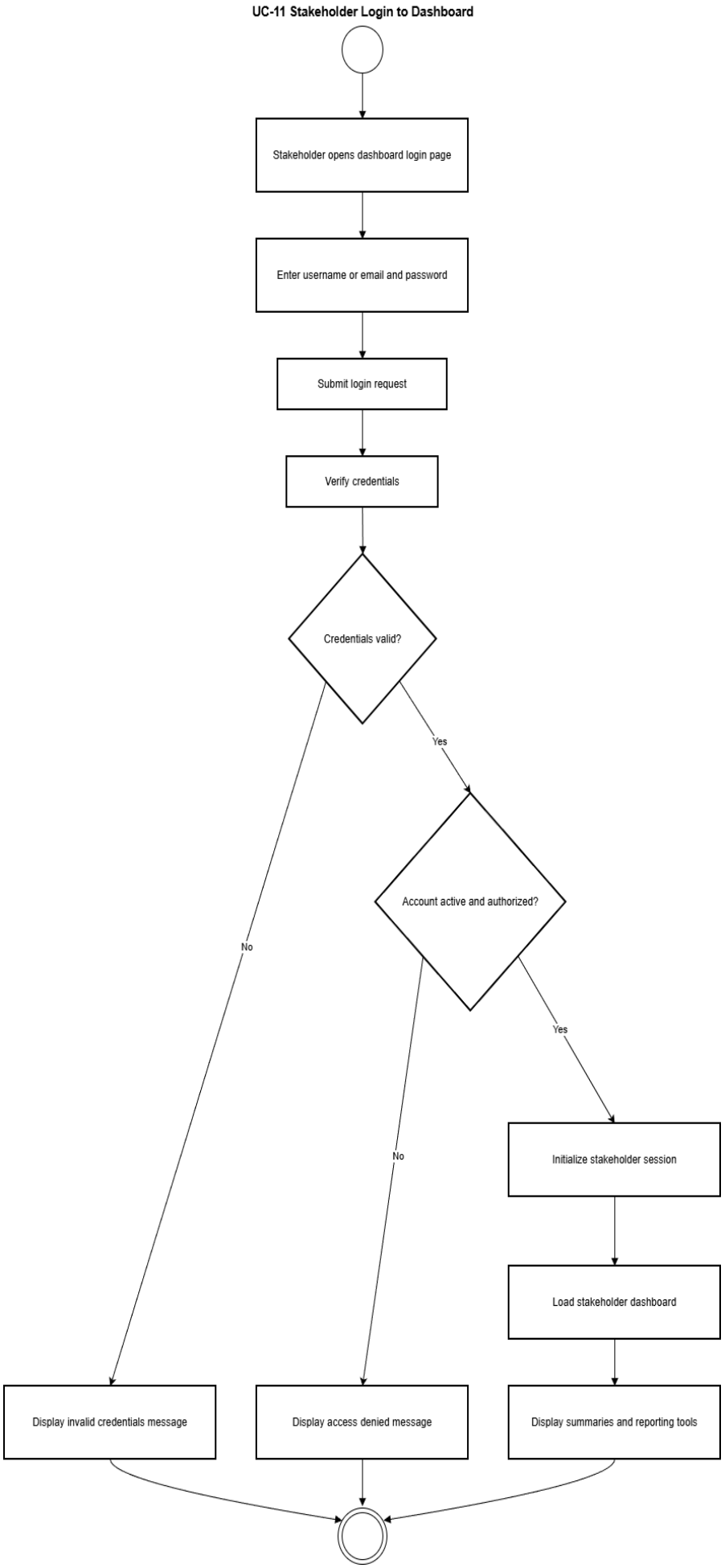


**UC-9 Receive Disease Diagnosis**

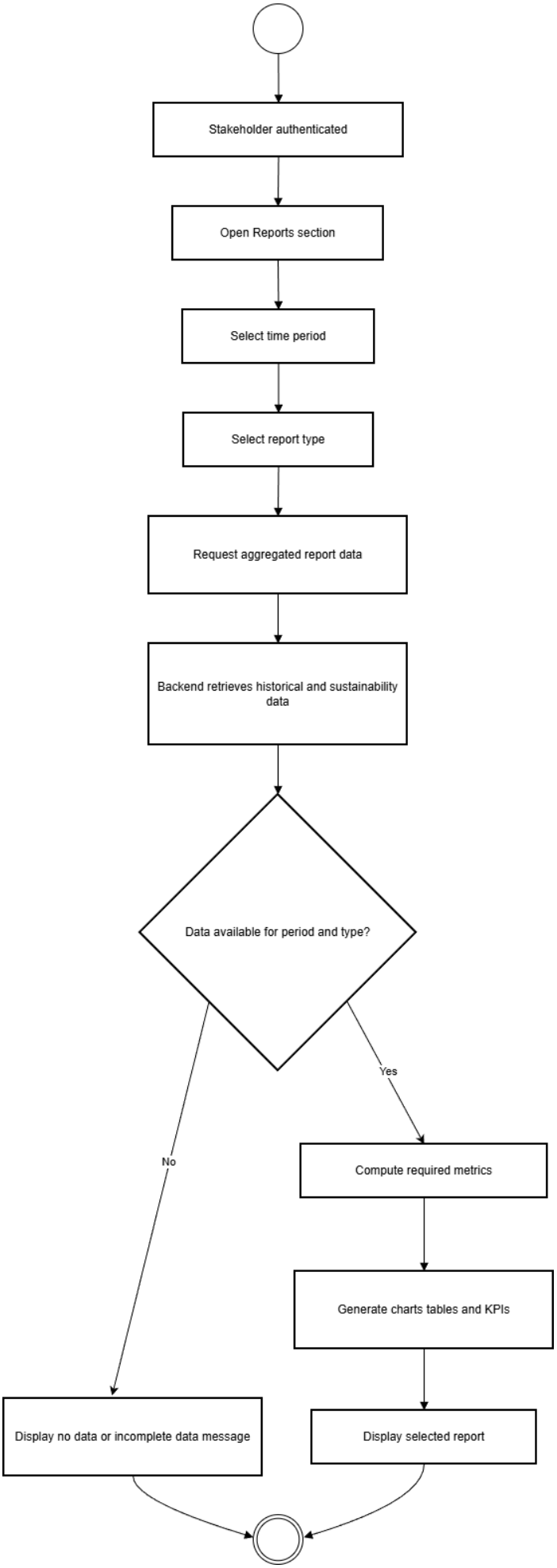


**UC-10 View Recommended Actions**

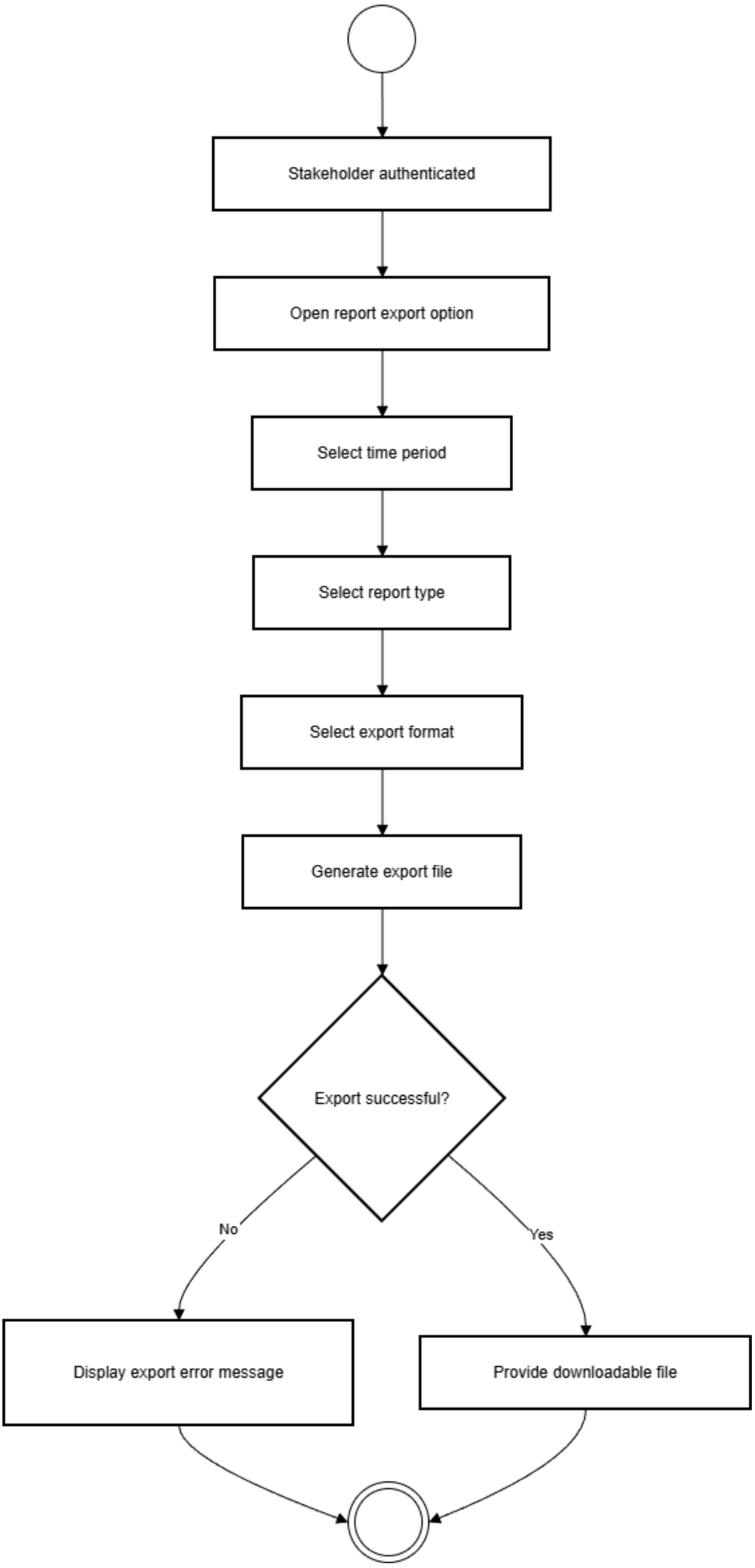



UC-11 Stakeholder Login to Dashboard
Stakeholder opens dashboard login page
Enter username or email and password
Submit login request
Verify credentials
Credentials valid?
Yes
No
Account active and authorized?
Yes
No
Initialize stakeholder session
Load stakeholder dashboard
Display invalid credentials message
Display access denied message
Display summaries and reporting tools

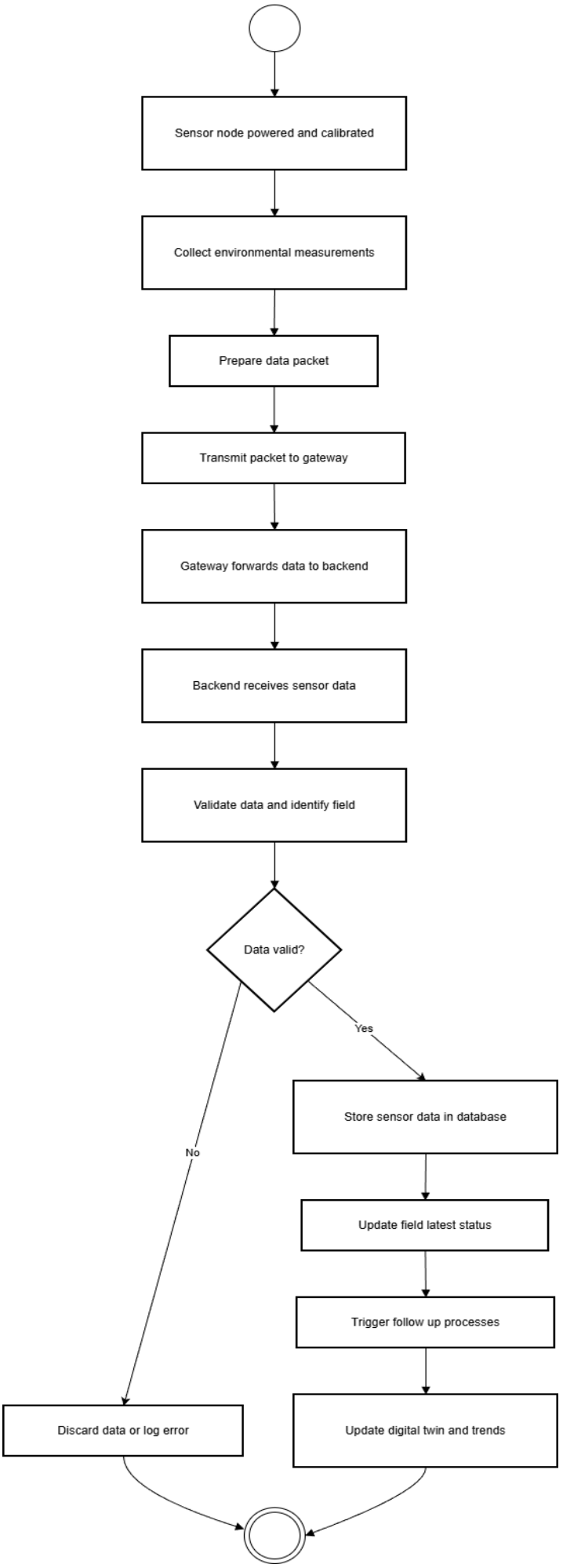
**UC-12 View Sustainability and Carbon Footprint Reports**



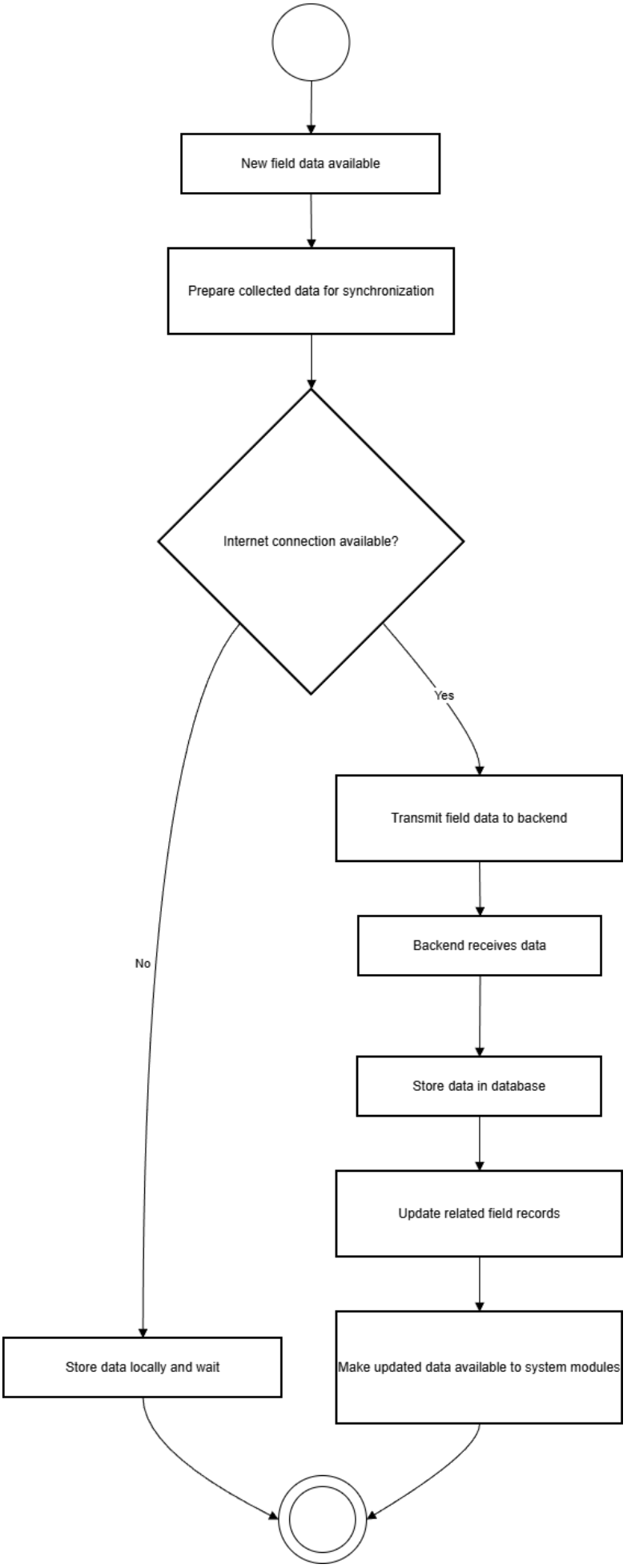
**UC-13 Export Sustainability and Carbon Footprint Reports**

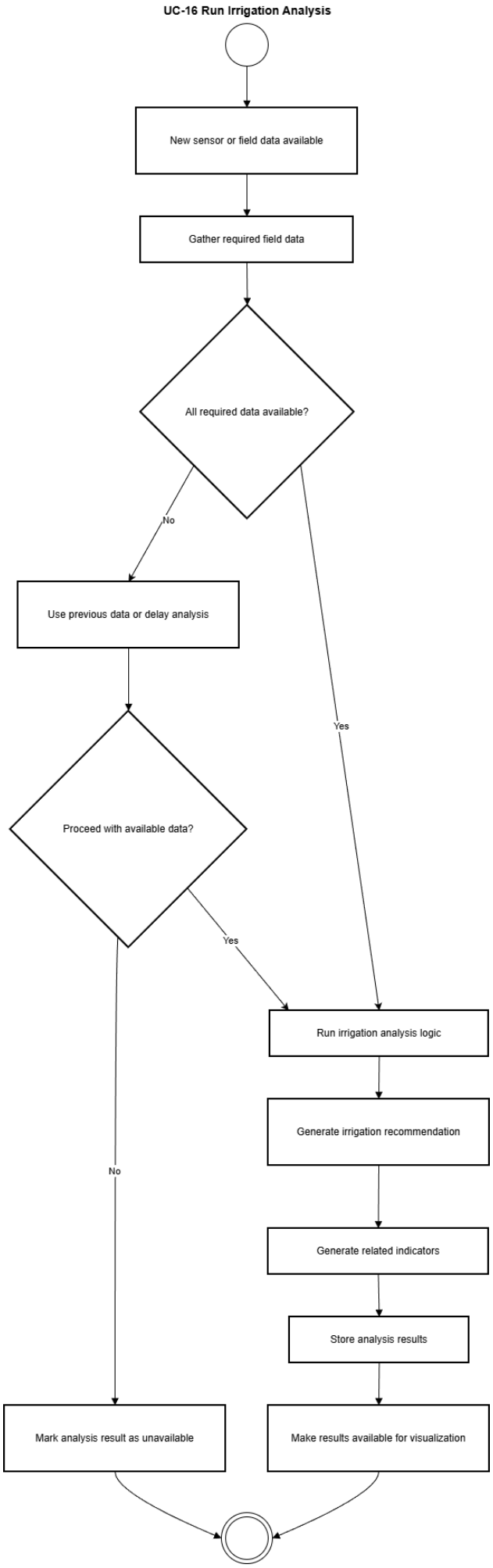


## UC-14 Collect and Transmit Sensor Data

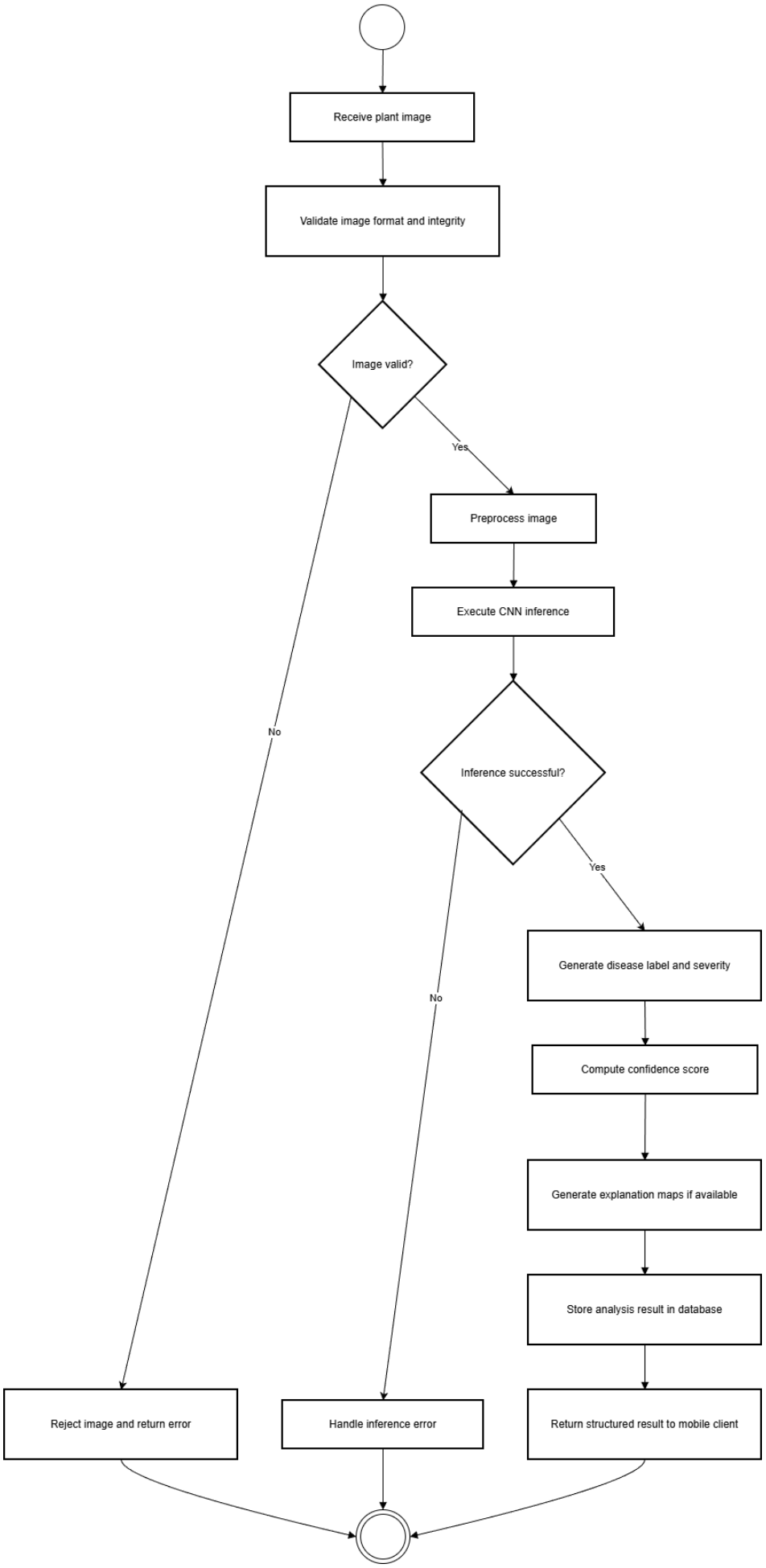


## UC-15 Sync Field Data to Backend

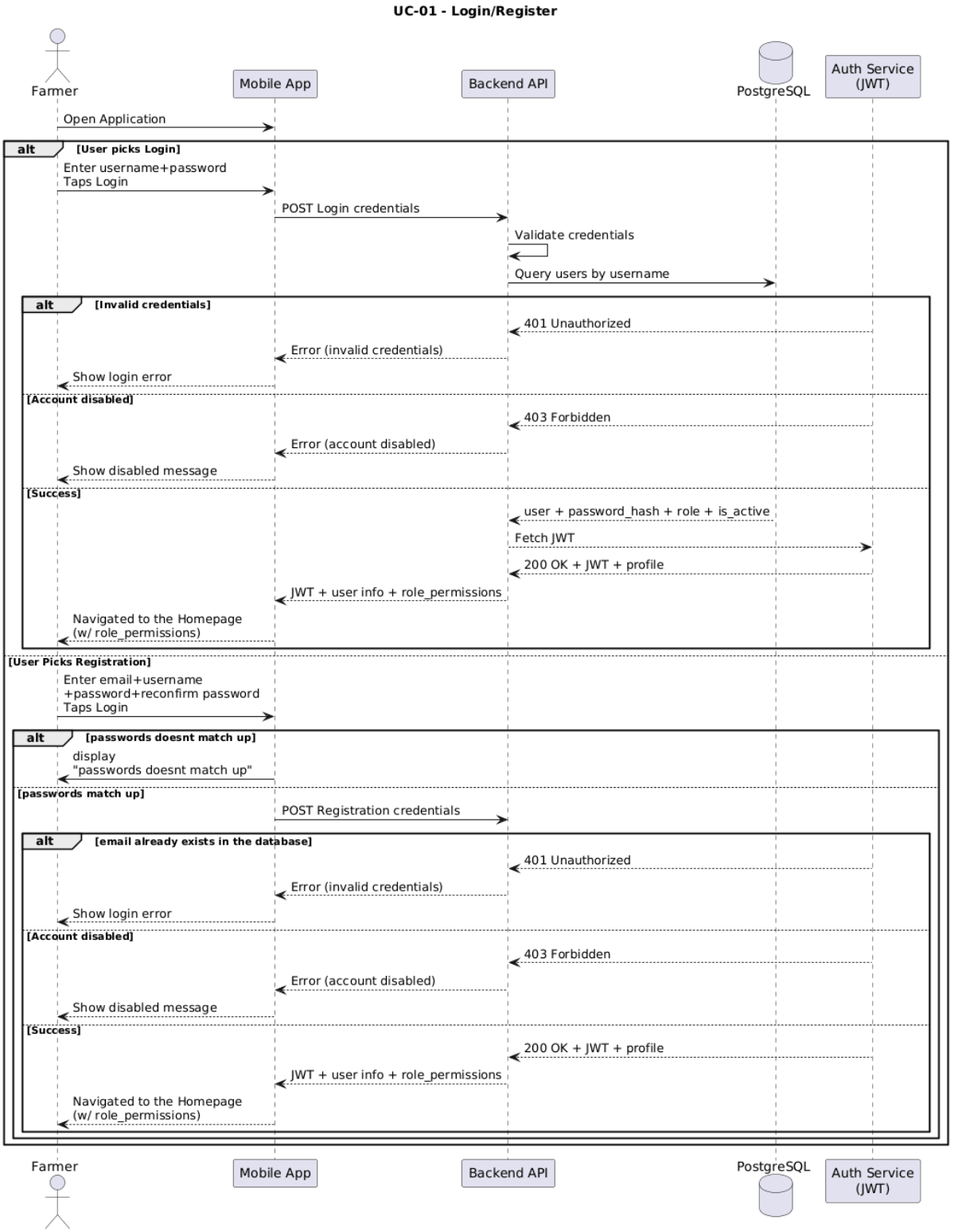


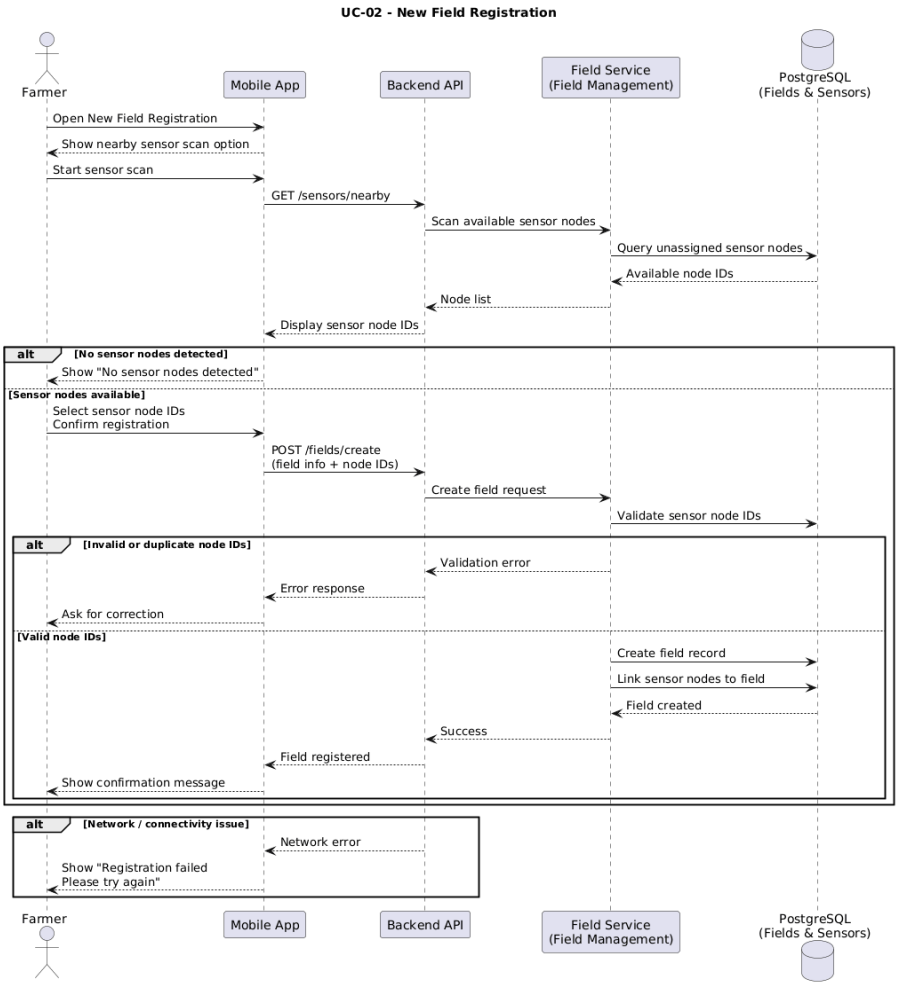

UC-16 Run Irrigation Analysis
New sensor or field data available
Gather required field data
All required data available?
No
Yes
Use previous data or delay analysis
Proceed with available data?
Yes
No
Run irrigation analysis logic
Generate irrigation recommendation
Generate related indicators
Store analysis results
Mark analysis result as unavailable
Make results available for visualization

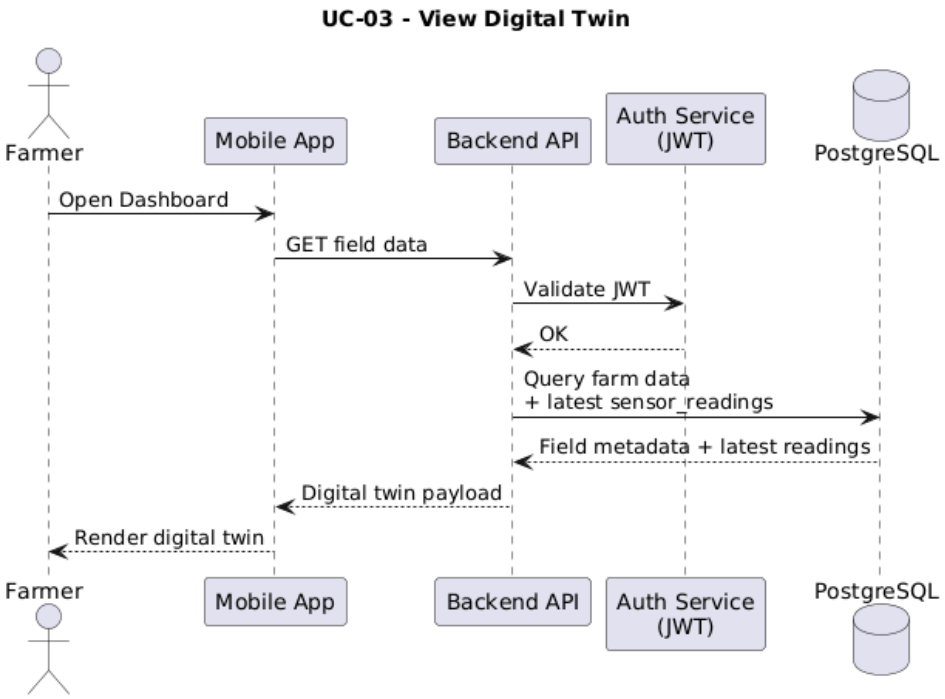
**UC-17 Analyze Plant Image for Diseases**



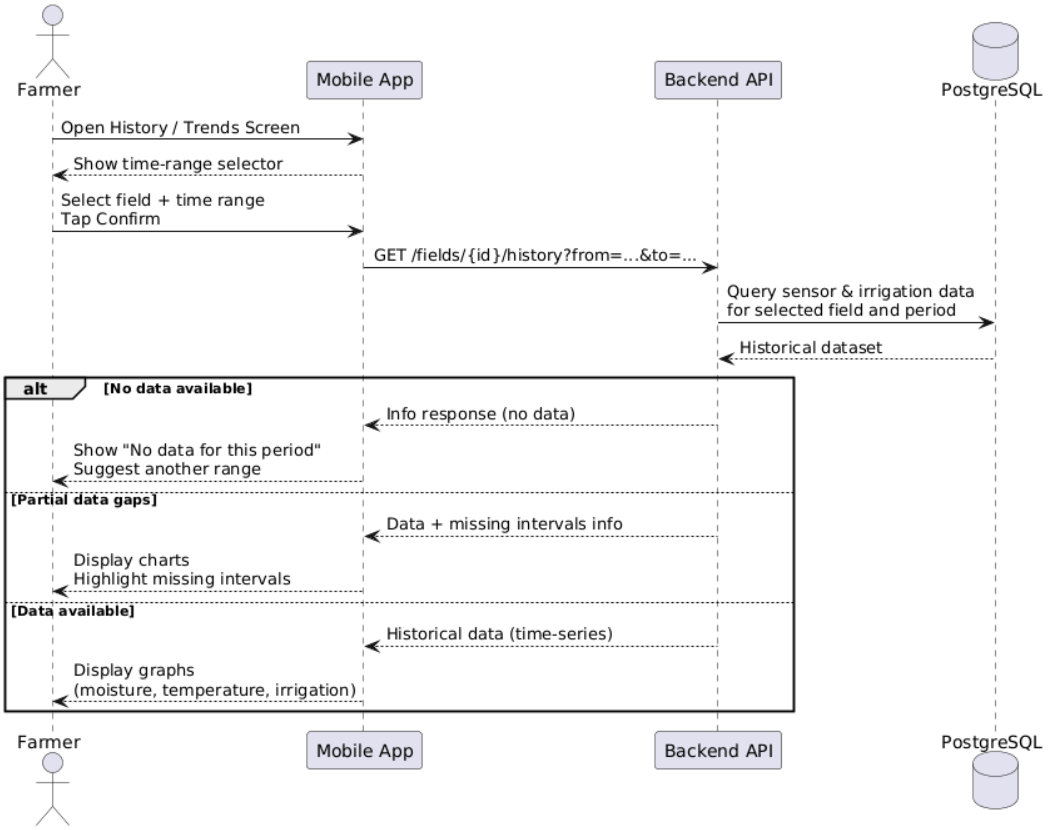
### 3.3.2. Sequence Diagrams



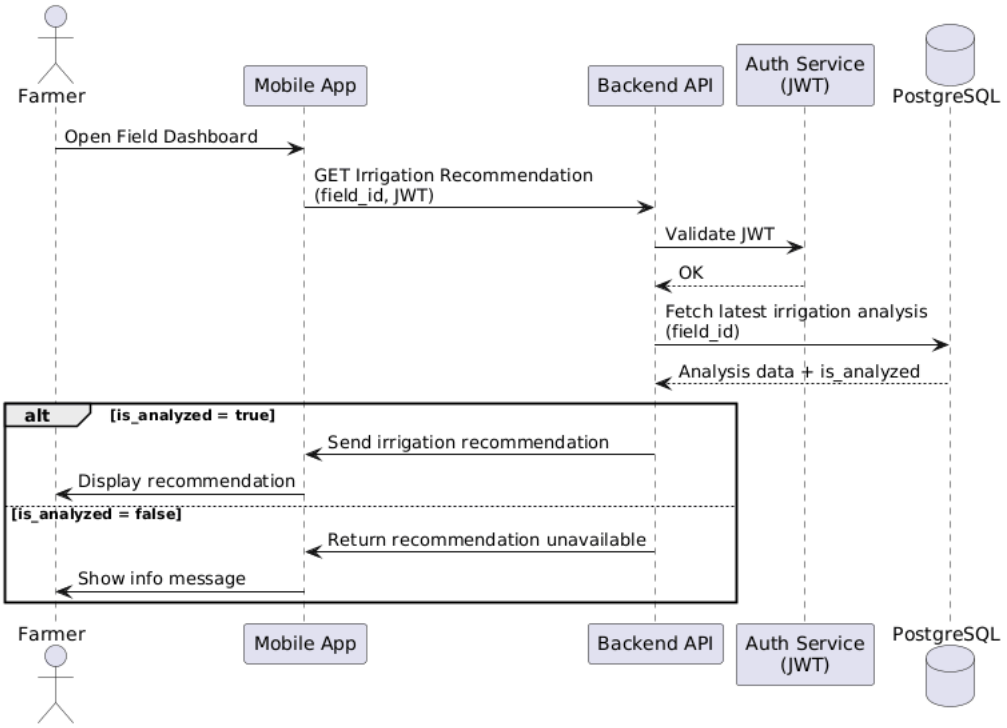

UC-02 - New Field Registration
Farmer
Mobile App
Backend API
Field Service
(Field Management)
PostgreSQL
(Fields & Sensors)
Open New Field Registration
Show nearby sensor scan option
Start sensor scan
GET /sensors/nearby
Scan available sensor nodes
Query unassigned sensor nodes
Available node IDs
Node list
Display sensor node IDs
alt
[No sensor nodes detected]
Show "No sensor nodes detected"
[Sensor nodes available]
Select sensor node IDs
Confirm registration
POST /fields/create
(field info + node IDs)
Create field request
Validate sensor node IDs
alt
[Invalid or duplicate node IDs]
Validation error
Error response
Ask for correction
[Valid node IDs]
Create field record
Link sensor nodes to field
Field created
Success
Field registered
Show confirmation message
alt
[Network / connectivity issue]
Network error
Show "Registration failed
Please try again"


UC-03 - View Digital Twin
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Dashboard
GET field data
Validate JWT
OK
Query farm data
+ latest sensor readings
Field metadata + latest readings
Digital twin payload
Render digital twin

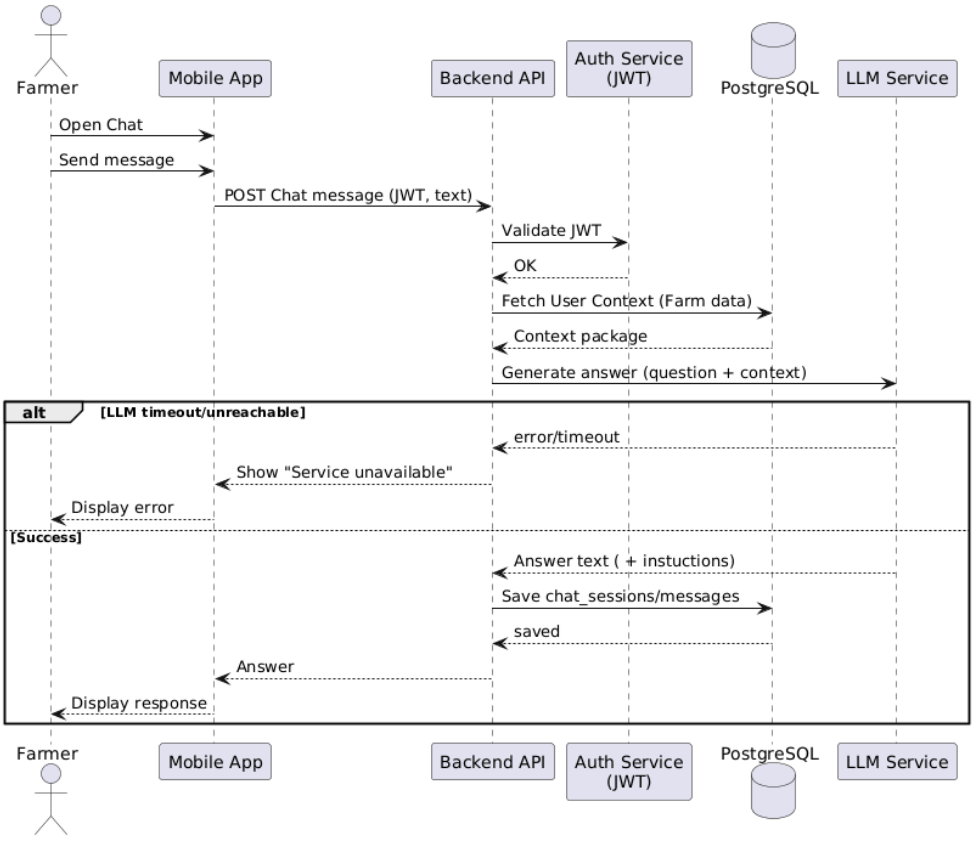
**UC-04 - View Historical Data / Trends**



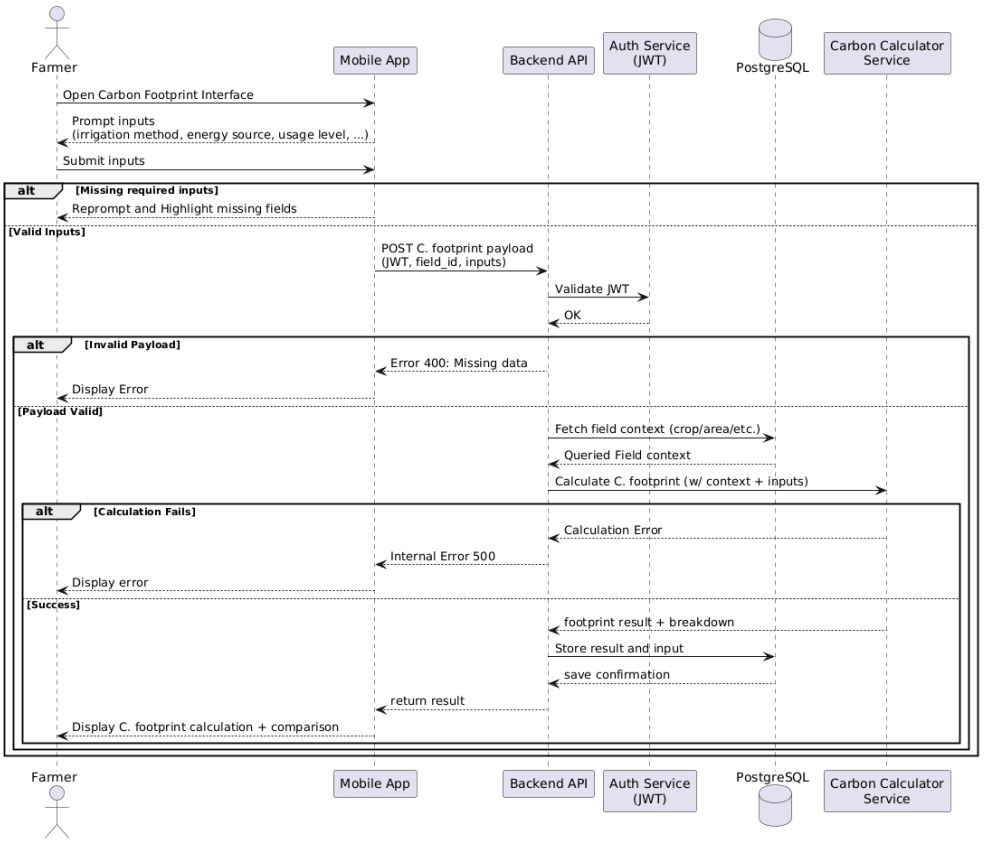

**UC-05 - Receive Irrigation Suggestion**



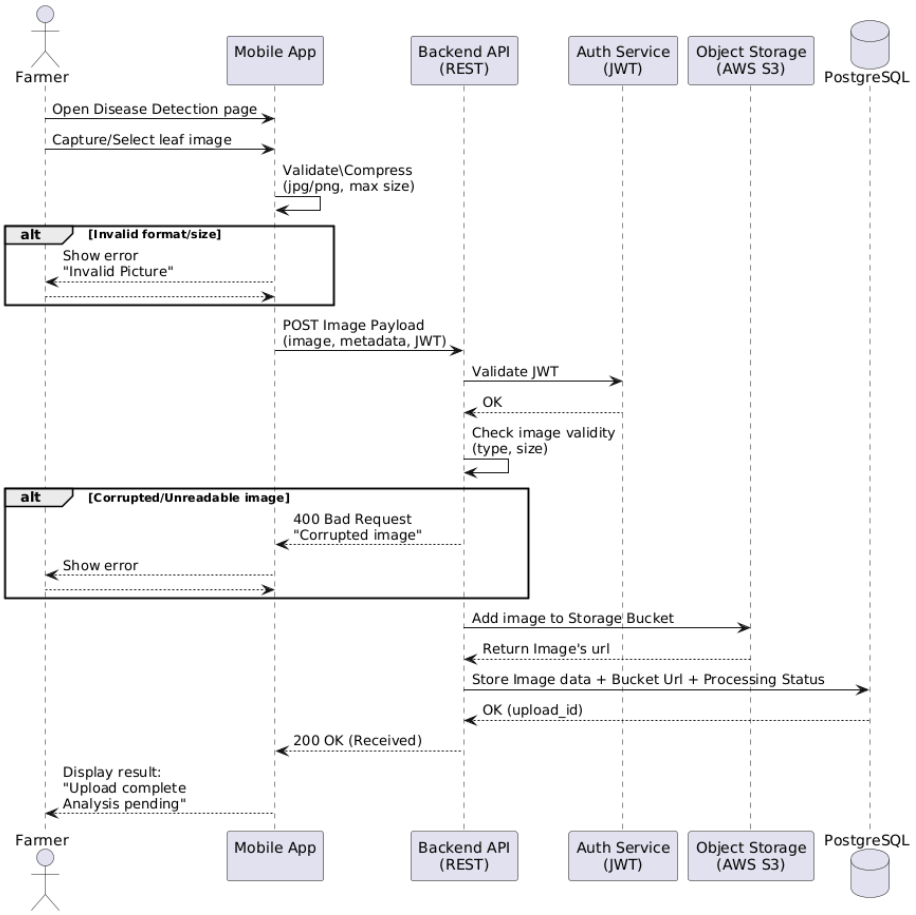
**UC-06 - AI Decision-Support Chatbot**



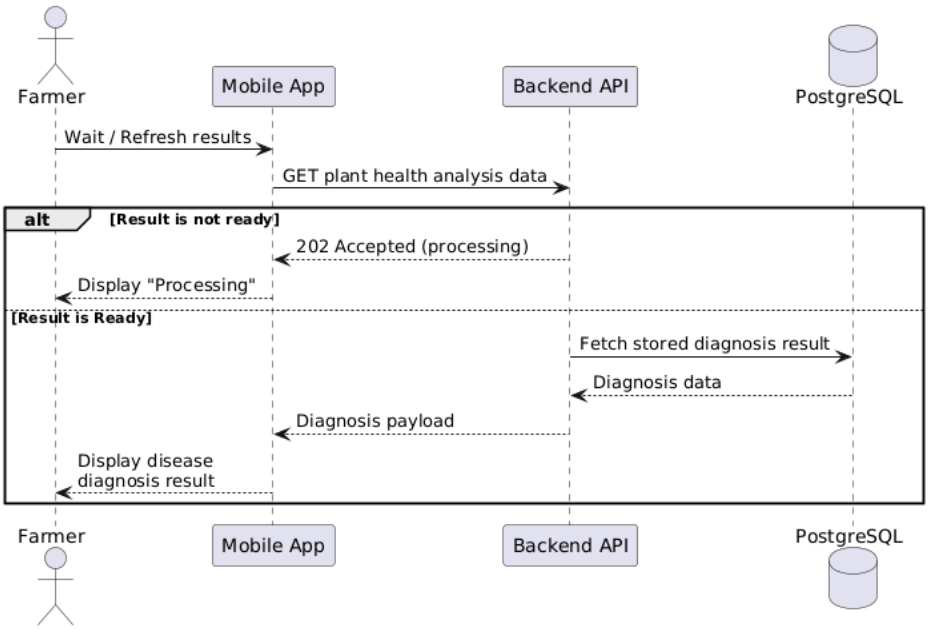

**UC-07 - Calculate Carbon Footprint**

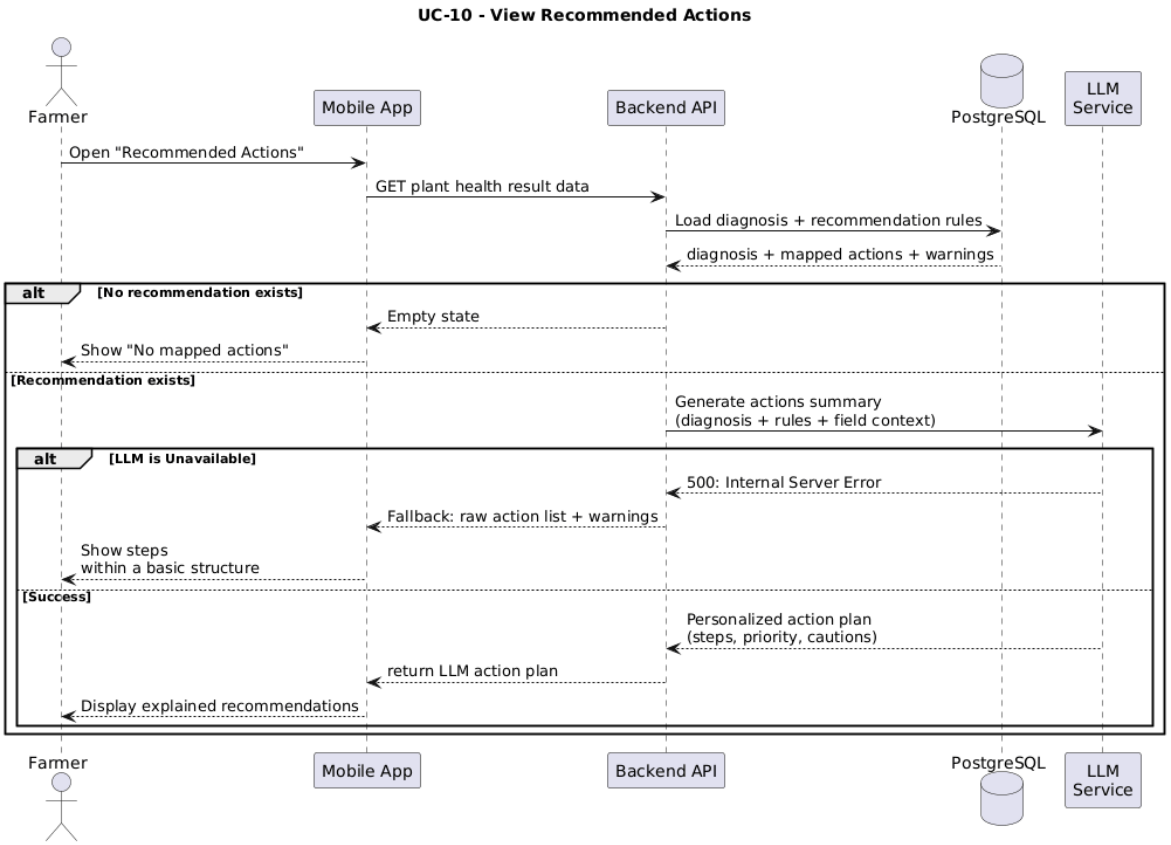


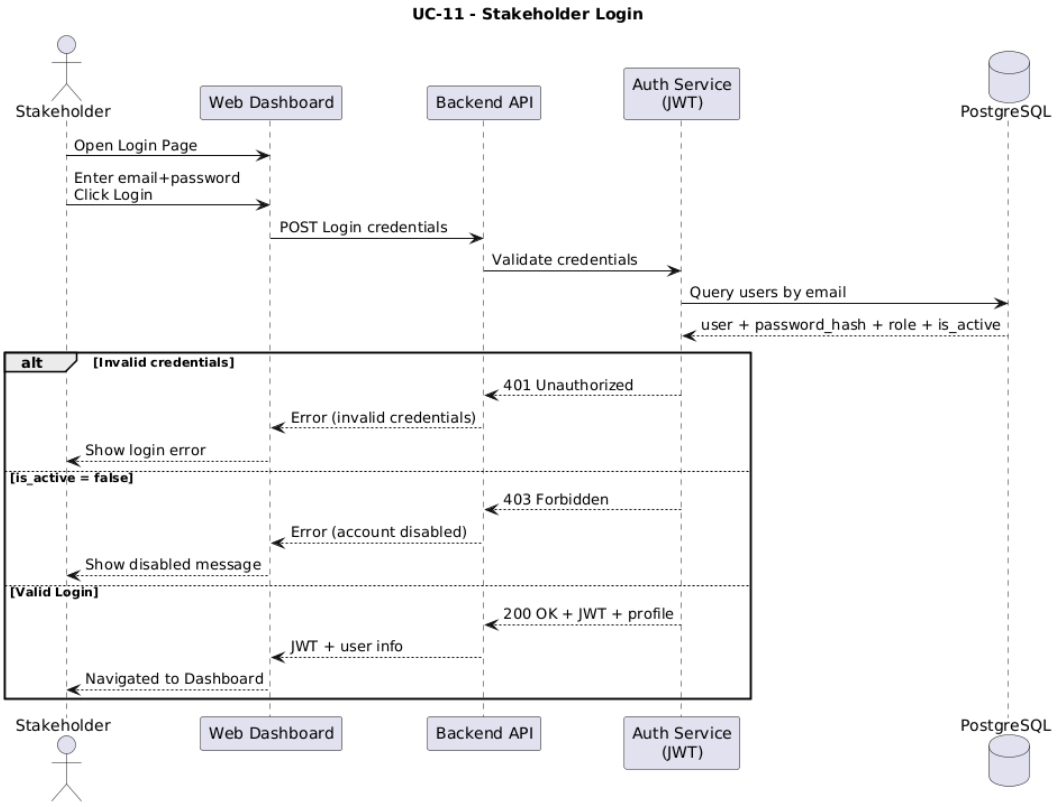
**UC-08 - Upload Plant Image**



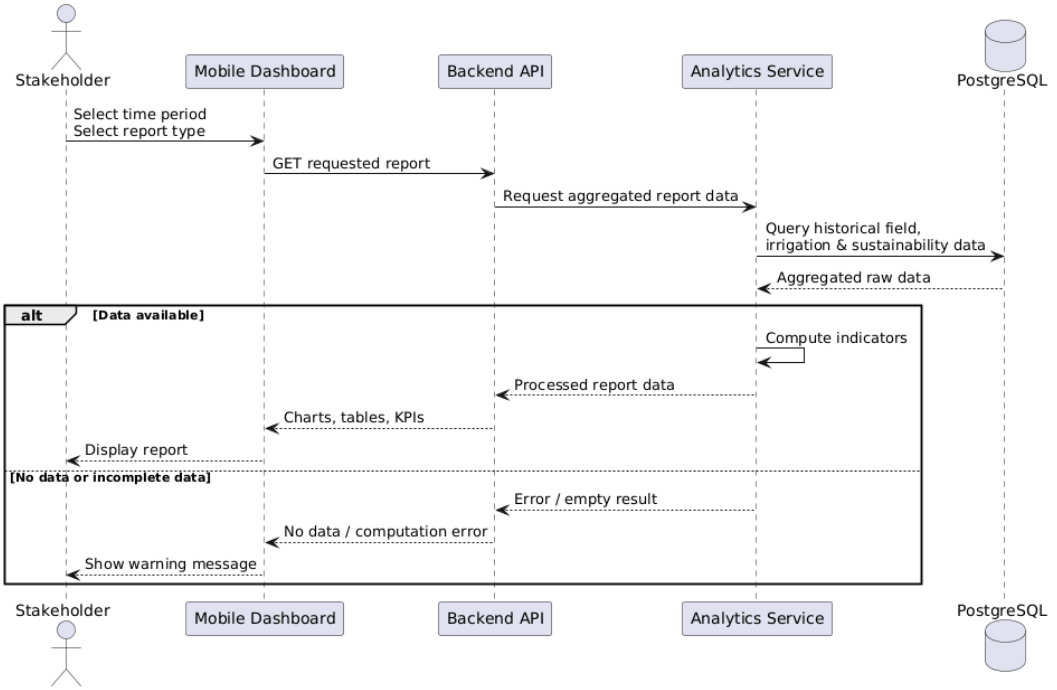

**UC-09 - Receive Disease Diagnosis**



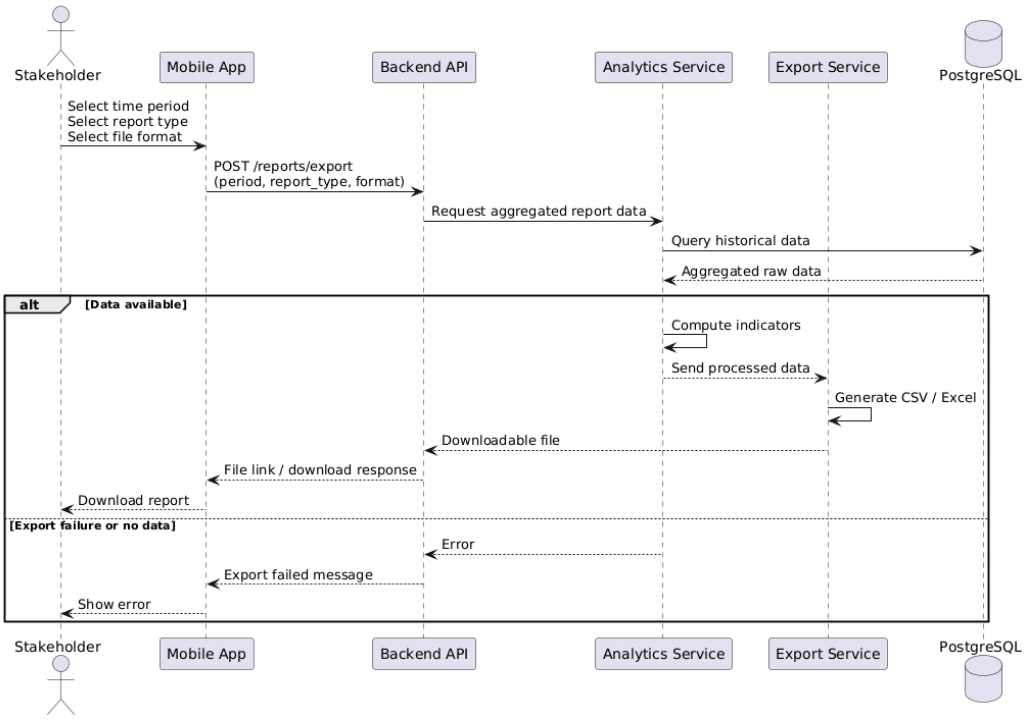

UC-10 - View Recommended Actions
Farmer
Mobile App
Backend API
PostgreSQL
LLM Service
Open "Recommended Actions"
GET plant health result data
Load diagnosis + recommendation rules
diagnosis + mapped actions + warnings
alt
[No recommendation exists]
Empty state
Show "No mapped actions"
[Recommendation exists]
Generate actions summary
(diagnosis + rules + field context)
alt
[LLM is Unavailable]
500: Internal Server Error
Fallback: raw action list + warnings
Show steps
within a basic structure
[Success]
Personalized action plan
(steps, priority, cautions)
return LLM action plan
Display explained recommendations


UC-11 - Stakeholder Login
Stakeholder
Web Dashboard
Backend API
Auth Service
(JWT)
PostgreSQL
Open Login Page
Enter email+password
Click Login
POST Login credentials
Validate credentials
Query users by email
user + password_hash + role + is_active
alt
[Invalid credentials]
401 Unauthorized
Error (invalid credentials)
Show login error
[is_active = false]
403 Forbidden
Error (account disabled)
Show disabled message
[Valid Login]
200 OK + JWT + profile
JWT + user info
Navigated to Dashboard

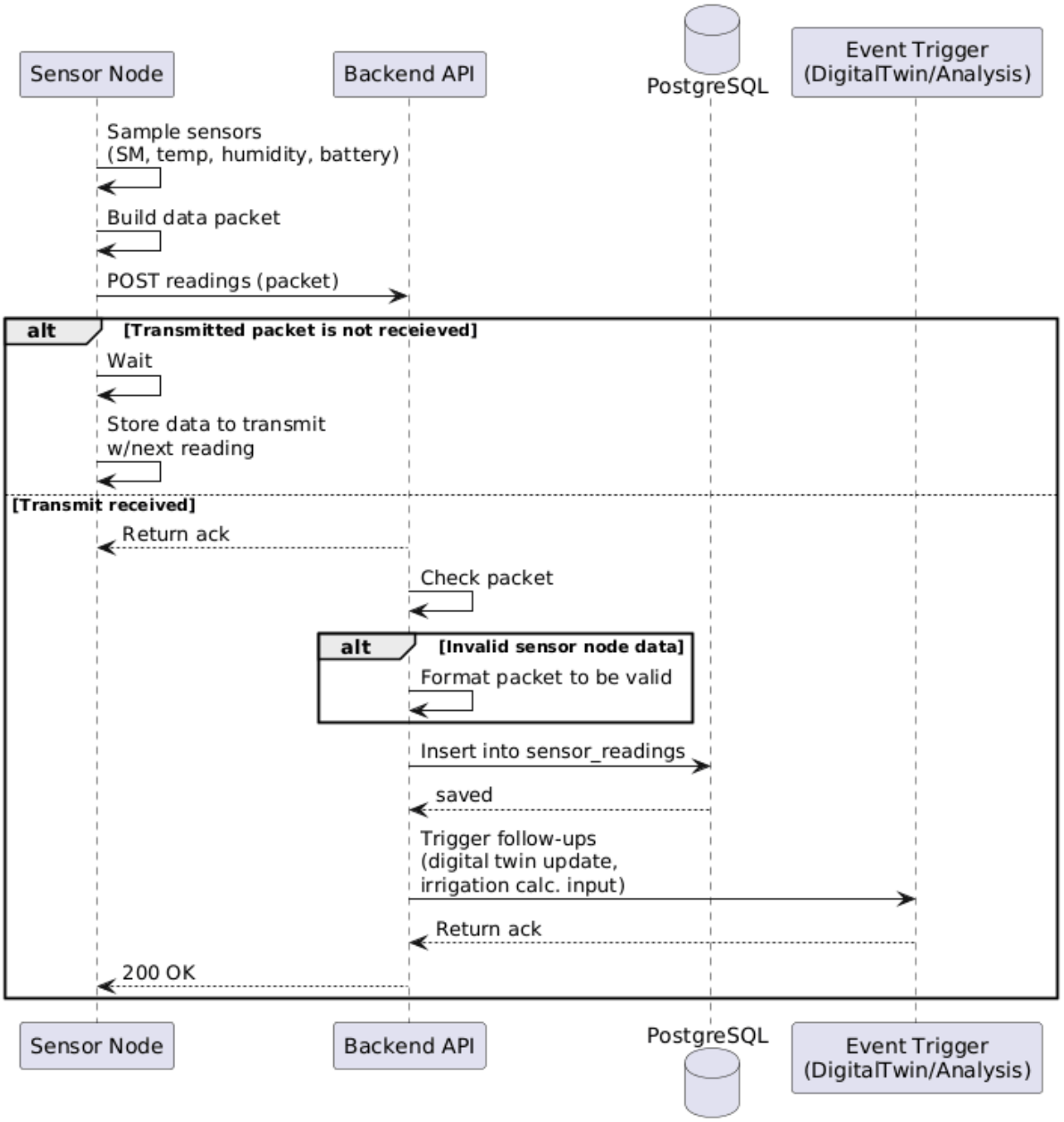
**UC-12 - View Sustainability and Carbon Footprint Reports**



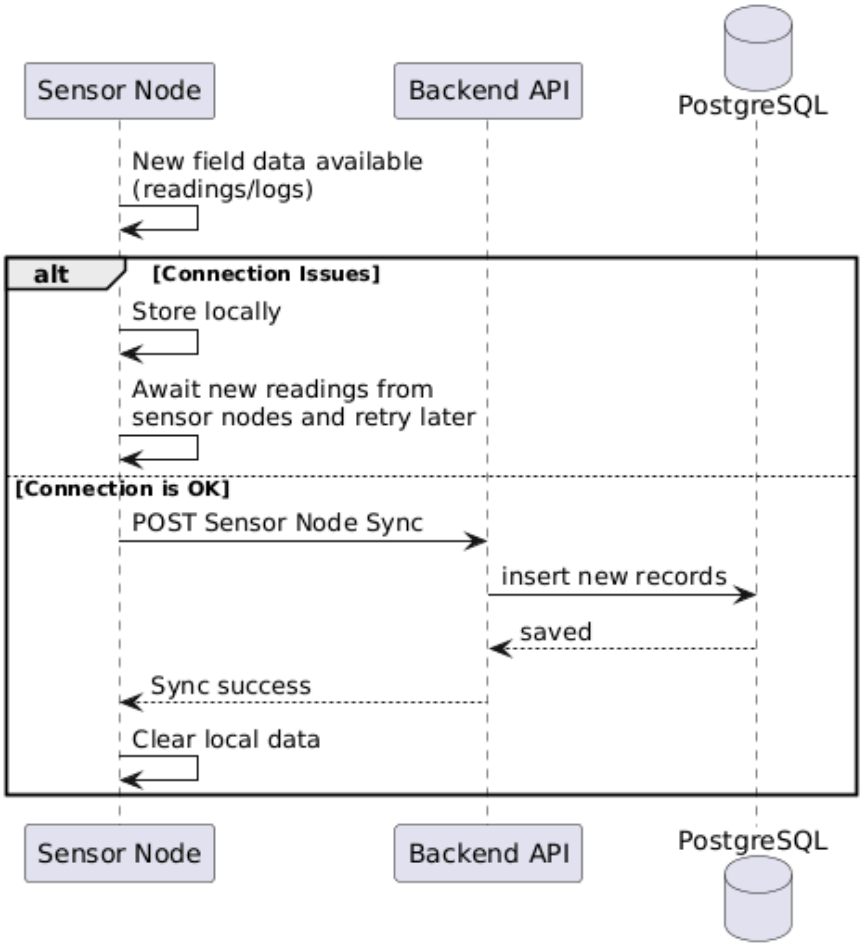

**UC-13 - Export Sustainability and Carbon Footprint Reports**



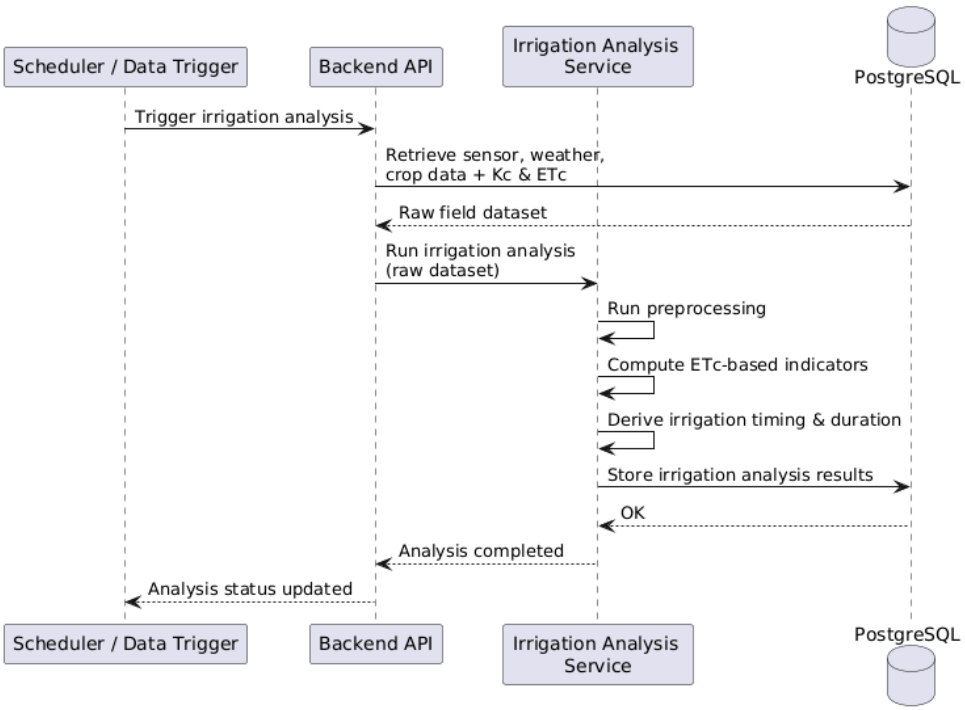
**UC-14 - Collect and Transmit Sensor Data**



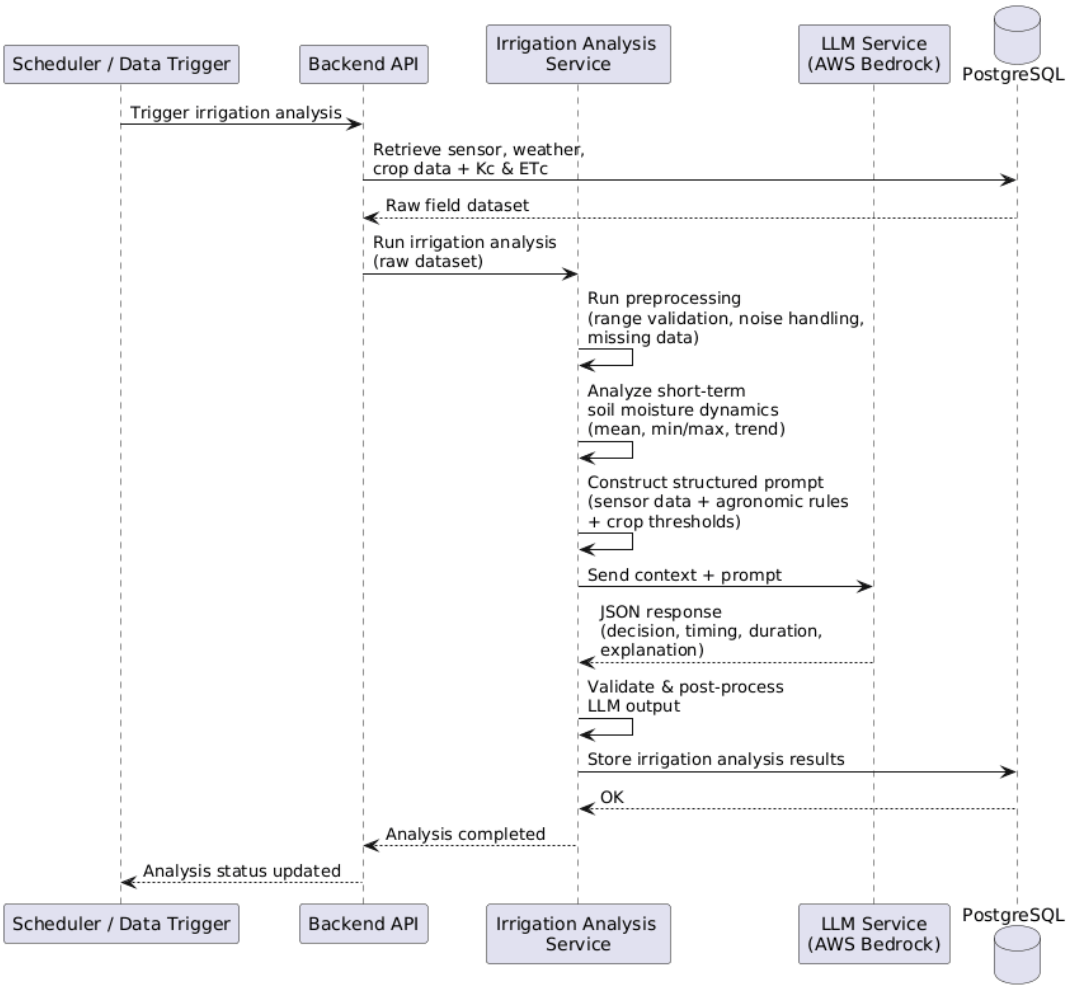
## UC-15 - Sync Field Data to Backend



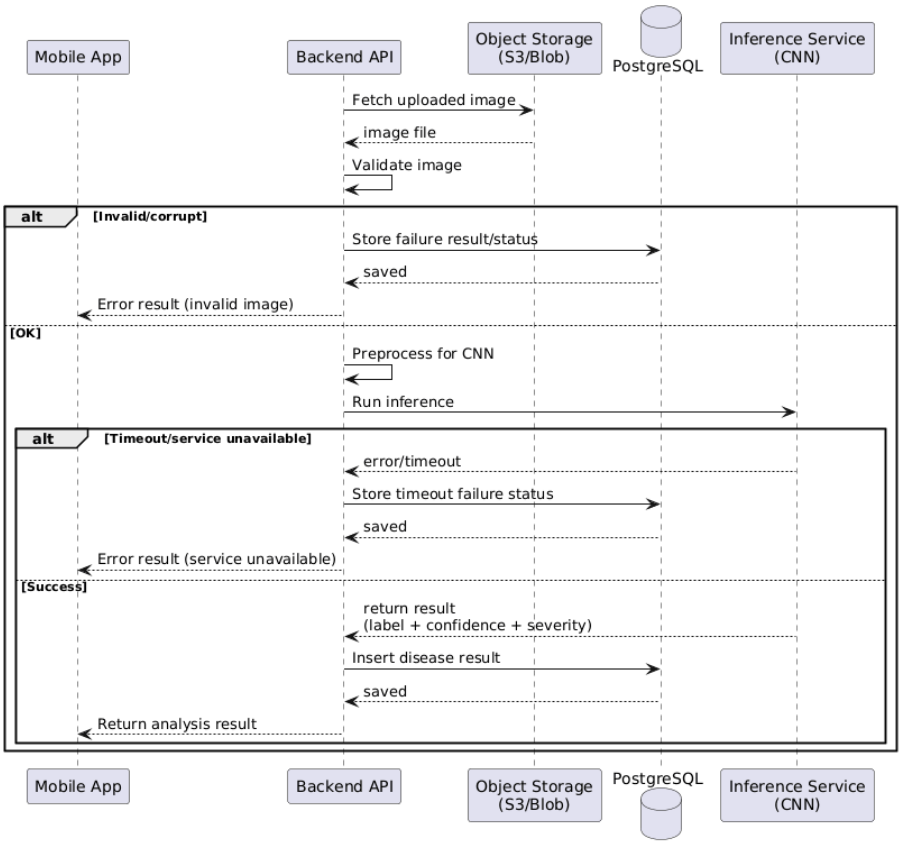

## UC-16 - Run Irrigation Analysis



**UC-16 - Run Irrigation Analysis**



**UC-17 - Analyze Plant Image for Diseases**



# 4. Methodology

## 4.1. Irrigation Prediction

This study does not rely on a traditional data-driven machine learning model for irrigation recommendation. Instead, the system adopts a hybrid approach that combines agronomy-based rules with Large Language Model (LLM)-driven reasoning.

The core irrigation logic is defined through a structured rule set derived from physical and agronomic principles, including soil moisture thresholds, and environmental conditions. These rules are embedded into a prompt template, which is provided to an LLM together with real-time sensor data.

The LLM acts as a decision engine that interprets the rule set and input data to generate structured irrigation recommendations in JSON format, including irrigation timing, duration or volume and explanatory reasoning. This approach ensures flexibility and explainability while avoiding the need for fully data-driven predictive modeling.

### 4.1.1. Problem Formulation

The irrigation recommendation problem is formulated as a rule-guided decision-making task that determines irrigation timing and duration based on environmental conditions, crop characteristics, and soil moisture dynamics.

Given crop-specific parameters, real-time sensor observations, and predefined agronomic rules, the objective is to decide whether irrigation should be initiated within a specified time window, and if so, to estimate the appropriate irrigation start time and duration.

Rather than directly computing irrigation schedules through deterministic equations or predictive models, these inputs and rules are structured into a prompt and evaluated by a Large Language Model (LLM). The LLM interprets the agronomic constraints and sensor data to generate a structured JSON output that includes irrigation decisions, timing, duration, and concise reasoning.

This formulation enables flexible, explainable, and context-aware irrigation recommendations while maintaining consistency with established agronomic principles.

### 4.1.2. Data Acquisition and Preprocessing

Data required for irrigation decision-making are collected from multiple sources and structured as inputs to the rule-based LLM prompt:

- Field sensor data, including soil moisture measurements as the primary variable, supported by air temperature and humidity readings to capture environmental conditions.
- Temporal soil moisture data, represented as short-term time-series (e.g., recent moisture values, trends such as increasing or decreasing, and summary statistics like minimum, maximum, and mean), which are used to assess moisture dynamics rather than single-point measurements.
- Crop and environment metadata, including crop type, soil type, irrigation method (e.g., drip)
- System parameters and thresholds, including crop-specific optimal moisture ranges, critical limits, and irrigation rules defined within the prompt template.
- Optional historical context, such as previous moisture behavior or recent irrigation patterns, when available, to improve decision consistency.

All inputs are standardized and embedded into a structured prompt, which is then evaluated by a LLM to generate irrigation recommendations in JSON format.

- Before irrigation decision-making, incoming data undergo lightweight validation and structuring to ensure consistency and reliability.
- Range validation, verifying that sensor values (e.g., soil moisture, temperature, humidity) fall within physically plausible limits.
- Basic noise handling, smoothing minor fluctuations in soil moisture readings when necessary.
- Missing data handling, where incomplete inputs are either inferred from recent values or flagged within the decision context.
- These steps do not constitute a full data-driven preprocessing pipeline but serve to ensure that the inputs provided to the LLM are consistent, interpretable, and aligned with the rule-based decision framework.

### 4.1.3. Algorithmic Approach

The irrigation recommendation process follows a rule-guided and LLM-driven workflow that ensures interpretability and flexibility:

- Input Aggregation: Real-time inputs, including soil moisture, environmental conditions (temperature, humidity), crop type, soil type, and irrigation method, are collected and structured.
- Temporal Analysis: Short-term soil moisture dynamics are analyzed using recent time-series data (e.g., mean, min/max values, and trend such as increasing or decreasing).
- Rule Encoding: Crop-specific thresholds (optimal range, critical limits) and irrigation heuristics are embedded into a structured prompt template.
- Context Construction: All inputs and rules are combined into a unified prompt that represents the current field state and decision constraints.
- LLM-Based Decision Making: The prompt is evaluated by a LLM, which interprets the rule set and input data to determine:
    - whether irrigation is required,
    - the recommended start time,
    - and the irrigation duration or water amount.
- Structured Output Generation: The LLM produces a standardized JSON output containing the irrigation decision, timing, duration, and concise explanation.

This approach replaces deterministic formula-based scheduling with a flexible, explainable decision layer while preserving agronomic constraints through explicit rule definitions.

### 4.1.4. System Evaluation Metrics

System performance is evaluated using operational and decision-oriented metrics rather than traditional ML measures:

- Decision Consistency: Agreement between irrigation recommendations and subsequent soil moisture behavior (e.g., whether recommended irrigation leads to recovery toward optimal ranges).
- Threshold Compliance: The extent to which soil moisture is maintained within predefined optimal and critical bounds after irrigation decisions.
- Over- and Under-Irrigation Reduction: Reduction in events where soil moisture exceeds upper limits or falls below critical thresholds.
- Recommendation Stability: Consistency of decisions across consecutive time windows under similar environmental conditions.
- Expert or Rule Alignment: Comparison of LLM-generated recommendations with predefined agronomic rules or expert expectations to ensure logical consistency.

Where historical irrigation data are available, recommended irrigation timing and duration can be compared with observed outcomes to provide validation.

### 4.1.5. Implementation Details

The irrigation decision logic is implemented as a modular service within the system architecture, designed to integrate rule-based reasoning with LLM-driven decision-making. The system is structured into distinct components to ensure maintainability and extensibility:

- Input Processing Module: Collects and formats real-time sensor data, environmental conditions and contextual metadata into a standardized structure.
- Rule and Prompt Engine: Encodes crop-specific thresholds, irrigation rules, and system constraints into a reusable prompt template.
- LLM Decision Module: Sends the structured prompt to a LLM, which evaluates the inputs and generates irrigation recommendations in a structured JSON format.
- Post-processing Module: Validates and interprets the LLM output, ensuring consistency with expected formats and safety constraints before use.

This modular design allows independent updates to rule definitions, prompt structures and model configurations without affecting the overall system pipeline, supporting future extensions such as additional crop types or adaptive rule refinement.

## 4.2. Disease Detection

### 4.2.1. Problem Formulation

This module is designed as an image processing and ML component of the TARAS system, focusing on the analysis of plant leaf images for disease identification. The primary objective is to enable users to capture or upload a leaf image, after which the system automatically detects the presence of plant disease and classifies its type.

Formally, the task is modeled as a multi-class image classification problem. Each input image **X** is mapped to a disease label **Y**, selected from a predefined set of categories representing both healthy and diseased plant conditions. The model aims to achieve high classification accuracy while ensuring robust generalization performance on previously unseen data.

### 4.2.2. Data Acquisition and Preprocessing

#### 4.2.2.1. Data Acquisition

Plant leaf images used in this study are primarily obtained from the publicly available PlantVillage dataset, which provides a large collection of labeled images under controlled conditions. To improve real-world applicability and model generalization, an additional dataset was constructed to better represent field conditions. This supplementary dataset was collected using a combination of automated image extraction tools (implemented in Python) and manual collection processes.

By combining the structured and balanced nature of the PlantVillage dataset with the variability of real-world images, the overall dataset aims to better capture practical conditions encountered in real deployments.

#### 4.2.2.2. Preprocessing

All input images are converted to RGB format to ensure consistency across different devices and acquisition conditions. Images are resized to a fixed resolution of 256 × 256 pixels to enable uniform processing.

To align the input distribution with ImageNet-pretrained EfficientNetV2 weights, pixel values are normalized using the corresponding preprocessing function.

To improve generalization, especially due to the domain gap between the controlled PlantVillage dataset and real-world images, data augmentation techniques are applied. These include random rotations, horizontal flips, slight zooming, and brightness variations, simulating diverse field conditions such as different lighting, orientations, and capture angles. This preprocessing pipeline ensures both computational efficiency and robustness under real-world deployment scenarios.

### 4.2.3. Algorithmic Approach

The disease detection module uses a deep learning–based image classification approach with a CNN, specifically EfficientNetV2-S, to identify plant diseases from leaf images.

Preprocessed images are passed through the CNN backbone, which extracts high-level visual features such as texture, color, and structural patterns. These features enable the model to distinguish between healthy and diseased leaves, as well as different disease categories.

A pretrained model is used to leverage general visual knowledge from large-scale datasets, improving robustness under varying real-world conditions such as lighting changes, background variability, and leaf orientation.

The final prediction is generated through a classification layer that maps extracted features to predefined disease classes, selecting the label with the highest probability.

### 4.2.4. Model Selection and Comparative Evaluation

Several CNN architectures, including MobileNetV3-Small, EfficientNet-B0, ResNet50, EfficientNet-B7, and EfficientNetV2-S, were considered for the disease detection module.

The selection was based on architectural efficiency and deployment suitability rather than empirical comparison. While lightweight models offer lower computational cost, and deeper models provide higher capacity, EfficientNetV2-S offers a balanced trade-off between accuracy and efficiency.

Therefore, EfficientNetV2-S is selected as the core classification model for the TARAS system.

### 4.2.5. System Evaluation Metrics

Model performance is to be evaluated using multiple complementary metrics to capture different aspects of classification quality:

- **Accuracy**: Overall proportion of correctly classified images
- **Precision**: Ability of the model to avoid false disease predictions
- **Recall**: Ability of the model to correctly identify diseased samples
- **F1-score**: Harmonic mean of precision and recall

### 4.2.6. Implementation Details

The plant disease detection module is planned to be implemented using TensorFlow and Keras. The model will be trained offline and later integrated into the TARAS system for inference.

The module is designed to classify preprocessed leaf images using an EfficientNetV2-S model and output the most probable disease label. Its modular structure allows independent updates and future improvements without affecting the overall system.

## 4.3. LLM-Based Advisory & Decision Support

### 4.3.1. Problem Formulation

The LLM-based advisory component aims to translate complex agricultural data into clear, context-aware decision support. While the system's predictive modules output raw numbers and categorical labels (such as disease types or moisture levels), farmers often need these results explained in plain language. Therefore, we formulate this as a natural language reasoning task: transforming structured system data into actionable, easy-to-understand agronomic recommendations.

To act as an intelligent bridge between the raw data and the user, the language model processes two main inputs: the real-time system context (soil moisture, irrigation thresholds, disease status) and the user's direct query. By combining these, it turns raw metrics into practical advice.

### 4.3.2. Input Representation and Context Construction

The LLM needs a well-structured input to provide accurate answers. We build this context by combining irrigation predictions, disease detections, and current environmental data.

To prevent the model from confusing background system data with the user's chat message, we isolate the information. The raw system outputs are formatted as JSON and placed inside specific XML-style tags (e.g., `<tarla_verisi>`). This background payload, along with the recent chat history, is fed to the model using conditional logic. The model is explicitly instructed to read this encapsulated data only when the user's question actually requires it. This targeted approach prevents the assistant from giving unprompted, robotic technical reports and keeps the conversation natural and relevant to the farmer's needs.

### 4.3.3. Response Generation and Explainability

The assistant generates actionable advice by explaining the system's findings in everyday language. It explicitly references key data points, such as current soil moisture or visual disease symptoms, to back up its statements. This approach builds user trust because it goes beyond simply telling the farmer *what* to do; it explains *why* the

recommendation is being made based on their specific field conditions.

### 4.3.4. System Evaluation and Limitations

Because the LLM acts as an interpreter rather than a prediction engine, evaluating it focuses on qualitative metrics: how closely it sticks to the provided data, how logical its answers are, and how useful the advice is to the farmer. The main goal is to maintain "zero-hallucination" boundaries—ensuring the model never invents facts.

However, the system has clear limitations. The assistant is only as reliable as the data it receives. If the IoT sensors or camera modules provide incorrect readings, the LLM will base its advice on that flawed data. Additionally, while the strict prompt prevents the AI from making things up, it also means the system might struggle to provide advice on situations it hasn't been programmed to handle, such as entirely new crop types or undocumented diseases. Therefore, the assistant is designed to be a helpful decision-support tool, engineered to assist—not replace—the judgment of expert agronomists.

### 4.3.5. Implementation Details

The LLM-based advisory module is engineered as a scalable API service, seamlessly integrated within the TARAS Node.js backend infrastructure.Communication between the mobile user interface and the backend is facilitated through RESTful API endpoints. Upon receiving a client request, the backend controller dynamically orchestrates context aggregation. It utilizes Prisma ORM to query the relational database (PostgreSQL) for real-time sensor metrics, dynamic agricultural thresholds, and the user's temporal session history. This aggregated context is serialized into a structured JSON payload and securely routed to the external LLM API endpoint. Finally, the generated natural-language advisory is parsed and returned to the frontend presentation layer, providing the end-user with immediate, actionable insights.

## 4.4. Sustainability & Carbon Footprint Calculation

### 4.4.1. Problem Formulation

This module calculates the carbon footprint of agricultural activities in the farm. The goal is to estimate how much greenhouse gas is produced during operations such as irrigation, energy use, fuel consumption and fertilizer application.

The problem is defined as converting activity data into carbon emission values. Each activity produces a certain amount of emissions based on its usage and a corresponding emission factor.

The system takes activity inputs and calculates total carbon emissions using a simple rule-based formula stated in 4.4.3 Calculation Approach.

The emission factors and calculation method used in this system are obtained from standardized sources such as IPCC and FAO, ensuring consistency and reliability in the calculations. This approach is transparent and does not rely on machine learning models.

### 4.4.2. Data Acquisition & Input Representation

The carbon footprint calculation module uses data collected from multiple sources within the system.

The main inputs include:

- Energy consumption data
- Fuel usage data
- Fertilizer application
- Water consumption
- Basic environmental data

These inputs are obtained from multiple sources, including, system logs and user inputs. In addition to automated data collection, the system also allows users to manually enter data based on external sources such as invoices or bills. For example, energy or water consumption can be recorded by the user when a bill is received. Fertilizer usage may also be entered manually by the user.

In addition to activity data, the system uses emission factors obtained from standardized sources such as IPCC and FAO. These values are stored within the system and matched with the corresponding activity type.

All inputs are converted into a consistent format before calculation. This includes unit standardization and simple validation to ensure values are within reasonable ranges.

This structure allows the system to combine data from different modules and represent them in a unified way for carbon footprint calculation.

### 4.4.3. Calculation Approach

The carbon footprint is calculated using a simple rule-based approach based on activity data and predefined emission factors. For each activity (such as energy use, fuel consumption or fertilizer application), the system calculates emissions using the following formula:

***Carbon Emissions = Activity Amount × Emission Factor***

Each activity is processed separately, and the total carbon footprint is obtained by summing all individual emissions. The calculation process follows these steps:

- Activity data is collected from system inputs (sensor data, user input or logs)
- The corresponding emission factor is selected based on the activity type
- Emissions are calculated for each activity
- All emissions are summed to obtain the total carbon footprint

The emission factors are predefined and stored in the system based on standardized sources such as IPCC and FAO.

This approach ensures that the calculation is straightforward, consistent and easy to interpret. It also allows the system to be easily extended by adding new activity types or updating emission factors when needed.

### 4.4.4. Output Metrics & Interpretation

The system presents carbon footprint results in a clear and simple way. The main output is the total carbon emissions (kg $CO_2$-equivalent), representing the overall environmental impact.

In addition, emissions are shown as a breakdown by activity type (such as, energy, fuel, fertilizer), helping users see which sources contribute the most.

Emissions can also be displayed in normalized forms, such as:

- per area
- per production amount (if available)

This allows users to better understand and compare their environmental impact.

### 4.4.5. Implementation Details

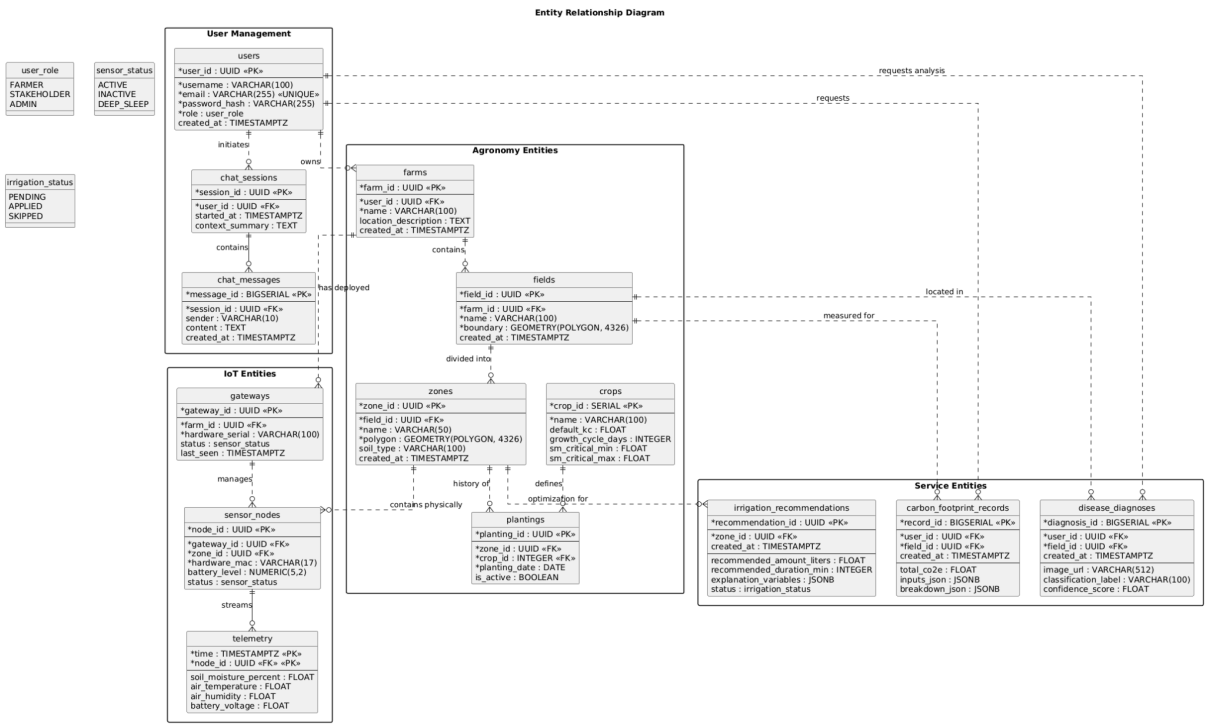
The carbon footprint calculation is implemented as part of the TARAS backend within the analytics layer. The system uses activity data from system logs and user inputs, along with predefined emission factors obtained from sources such as IPCC and FAO. Before calculation, input values are standardized into common units and checked for basic validity.

Carbon emissions are calculated using a rule-based formula for each activity, and the results are aggregated to obtain the total footprint. The calculated outputs are then stored and sent to the user interface through backend services, where they are displayed together with other system outputs.

This implementation allows the system to integrate carbon footprint results with existing modules such as irrigation and user data without requiring additional complex processing.

# 5. Data Design

## 5.1. Entity-Relationship Diagram



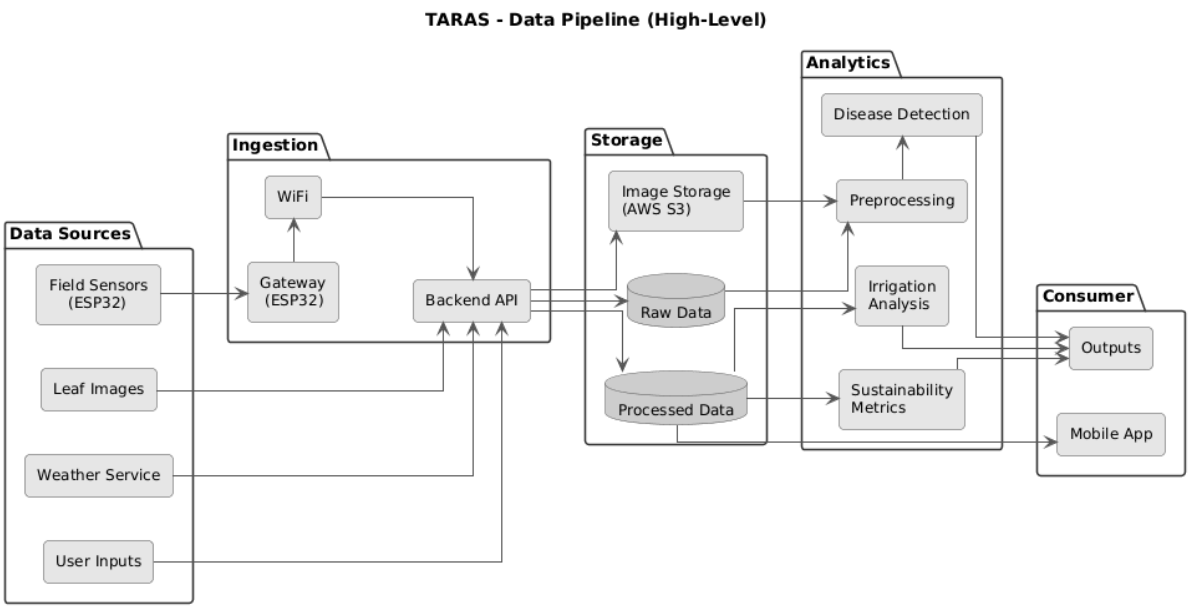
## 5.2. Data Pipeline

The TARAS data pipeline defines how heterogeneous data collected from field sensors, user inputs, and image uploads are transformed into reliable and analysis-ready information. The pipeline is designed to support real-time monitoring, historical analysis, and decision-support operations while maintaining data consistency and traceability.

Field sensor measurements such as soil moisture and environmental conditions are either transmitted to gateway devices first, or depending on the setup, directly forwarded to cloud services. In parallel, crop metadata, irrigation parameters and plant images are received through backend interfaces. All incoming data are timestamped and associated with field and device identifiers to ensure coherent integration.

Before storage, the data undergo preprocessing steps including timestamp alignment, unit normalization, basic quality checks, and context-aware filtering to handle missing values and abnormal sensor readings. Processed numerical data are stored in a centralized database, while image data are stored in dedicated storage buckets with linked metadata.

The processed data are then consumed by analytical modules responsible for irrigation estimation, disease detection, and sustainability analysis. Derived outputs such as irrigation recommendations, classification results and environmental indicators are stored as processed records and delivered to the mobile application through backend APIs. This structured pipeline enables consistent data flow across sensing, analytics and visualization layers and provides a scalable foundation for the TARAS platform.




# 6. User Interface Design

## 6.1. Interface Hierarchy

The interface follows a hierarchical structure that prioritizes critical agricultural information over details. Core metrics such as soil moisture, temperature and field status are positioned at the highest level of the hierarchy and are intended to be presented through large, clearly separated data cards. This structure enables users to access essential information with minimal navigation and reduces cognitive effort during decision-making.

The hierarchy is intentionally simplified to accommodate the diverse demographic profile of farmers, particularly older individuals with limited technical experience. By minimizing nested menus and reducing visual competition between interface elements, the interface supports rapid identification of actionable insights without overwhelming the user.

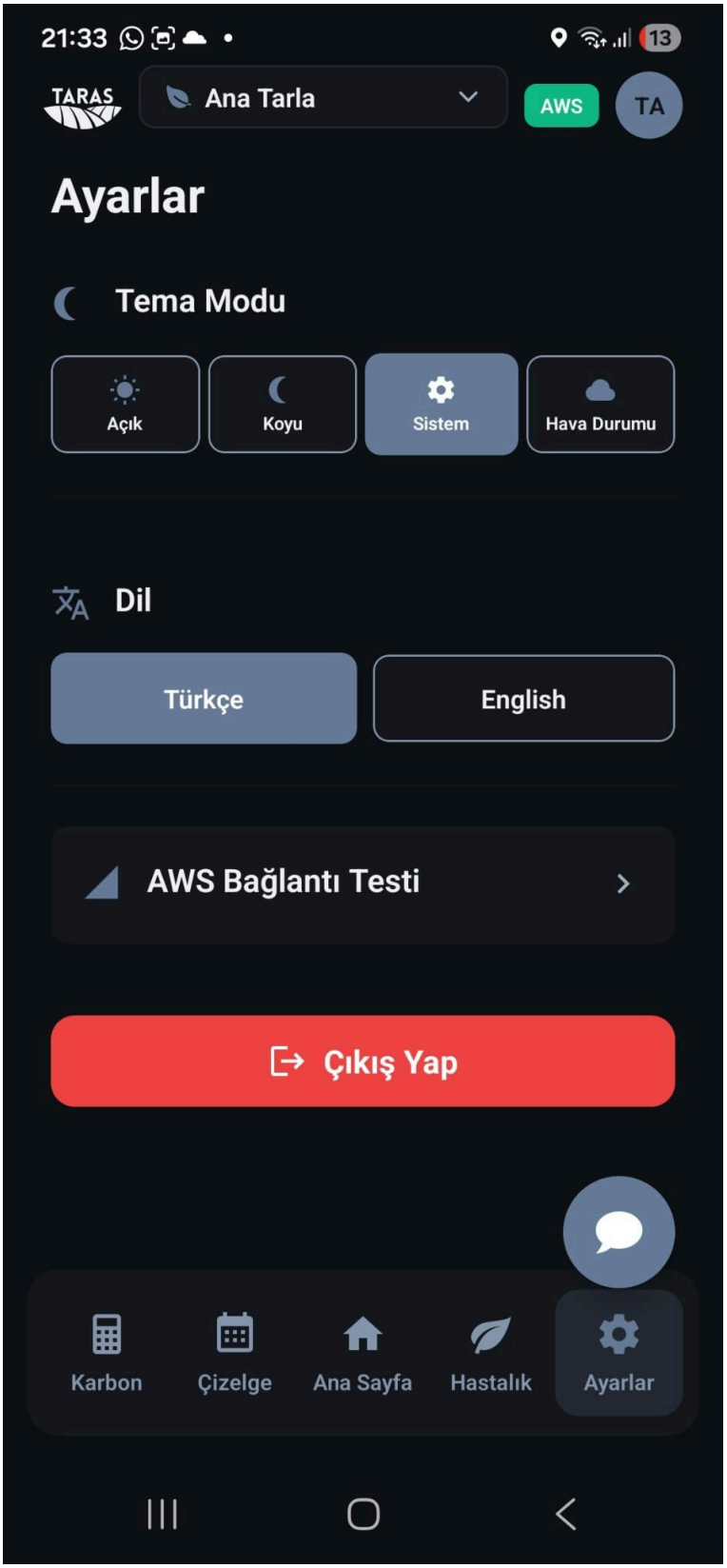
## 6.2. Visual Presentation

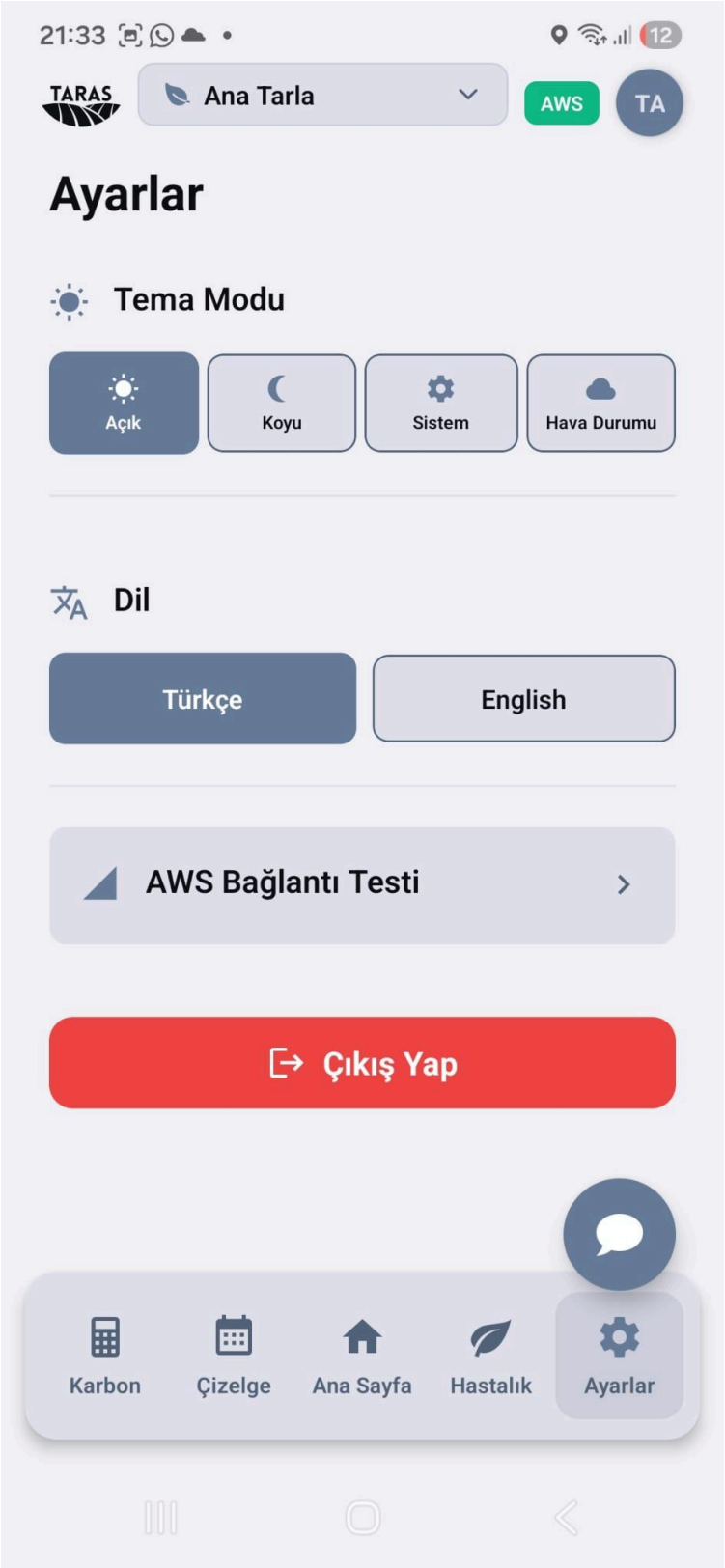
A balanced and minimalistic color palette was adopted as the foundation of the visualization and interaction layer. The palette is deliberately kept simple so that digital twin visualizations remain visually distinct from the interface background across different contexts. Light and dark modes are defined to assign inverse roles to background and text elements, preserving foreground and background separation, and visual comfort under varying lighting conditions.

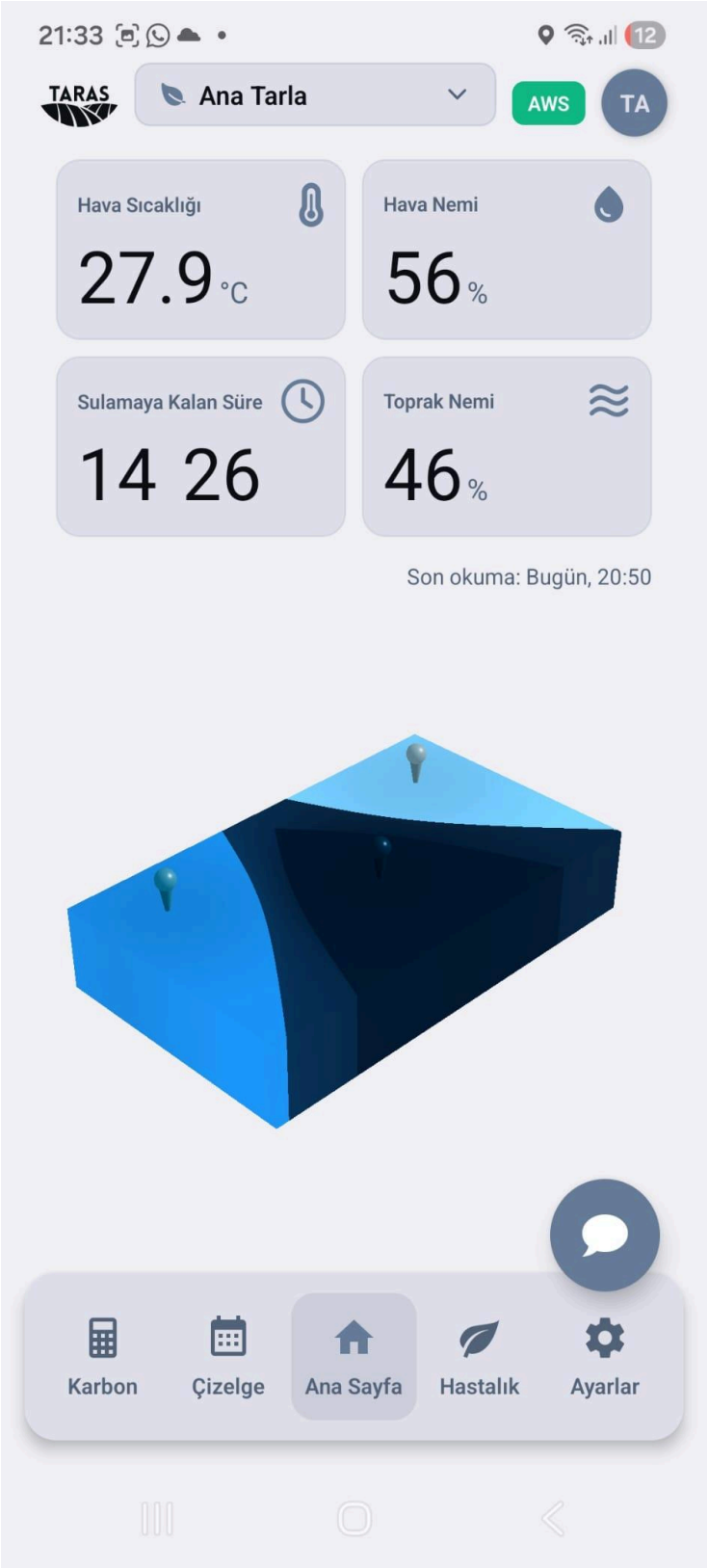
## 6.3. Accessibility and Responsivity

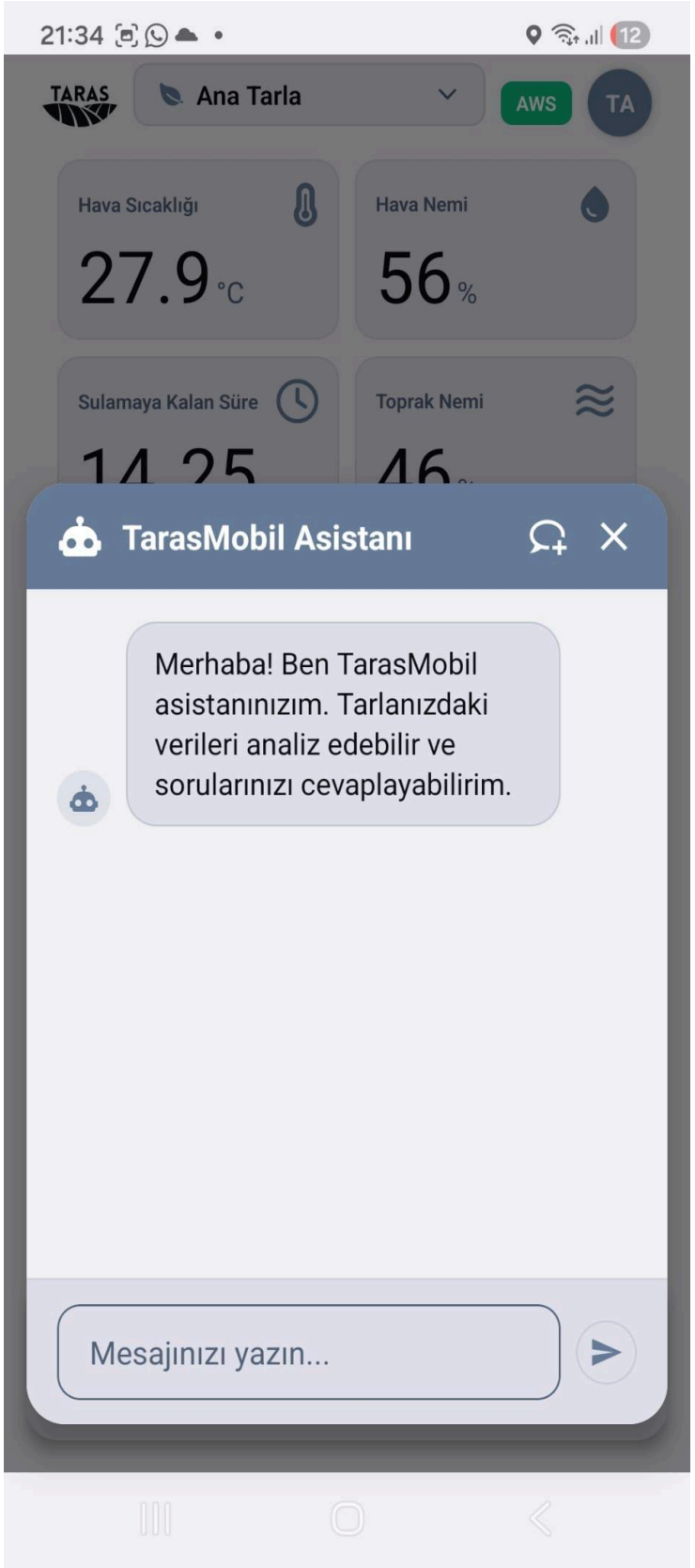
The interface design adheres to web accessibility and readability principles, including WCAG contrast recommendations, to support usability across varying lighting conditions and display settings. Text sizes, spacing, and contrast levels are specified to compensate for age-related declines in visual acuity and cognitive flexibility.

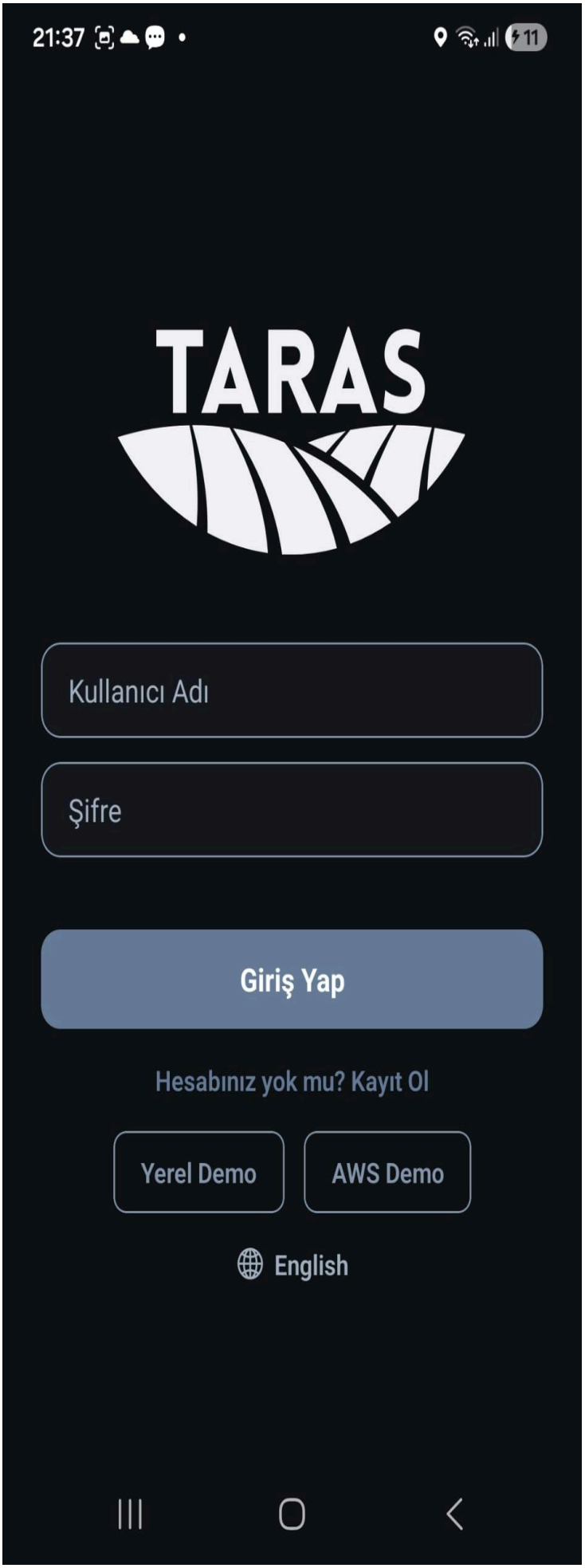
To further reduce cognitive load, essential information is structured to be presented in a concise and visually separated manner. In addition, an AI-powered conversational assistant is planned as a core interaction mechanism to support users with varying levels of technological literacy [5].

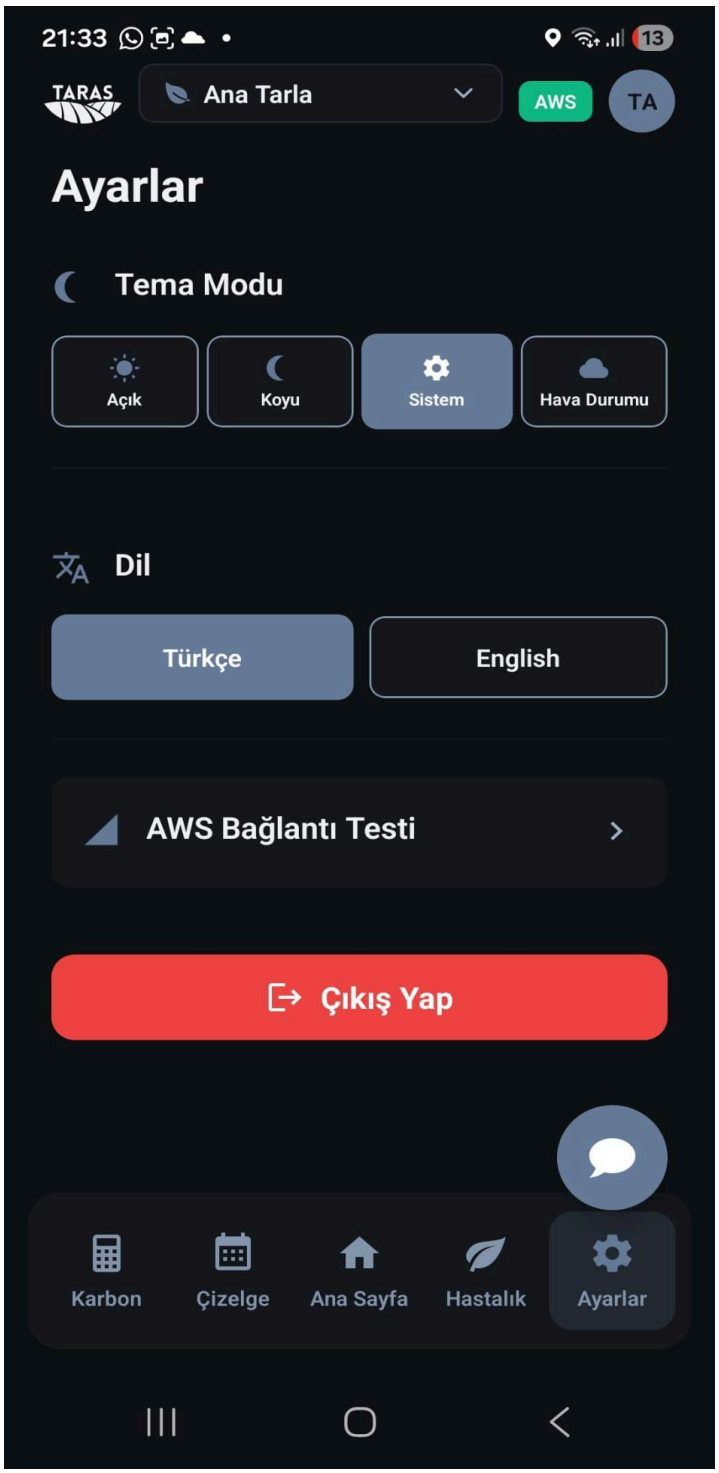

21:33
13
TARAS
Ana Tarla
AWS
TA
Ayarlar
Tema Modu
Açık
Koyu
Sistem
Hava Durumu
Dil
Türkçe
English
AWS Bağlantı Testi
Çıkış Yap
Karbon
Çizelge
Ana Sayfa
Hastalık
Ayarlar


21:33
12
TARAS
Ana Tarla
AWS
TA
Ayarlar
Tema Modu
Açık
Koyu
Sistem
Hava Durumu
Dil
Türkçe
English
AWS Bağlantı Testi
Çıkış Yap
Karbon
Çizelge
Ana Sayfa
Hastalık
Ayarlar


21:33
TARAS
Ana Tarla
AWS
TA
Hava Sıcaklığı
27.9 °C
Hava Nemi
56 %
Sulamaya Kalan Süre
14 26
Toprak Nemi
46 %
Son okuma: Bugün, 20:50
Karbon
Çizelge
Ana Sayfa
Hastalık
Ayarlar


21:34
TARAS
Ana Tarla
AWS
TA
Hava Sıcaklığı
27.9 °C
Hava Nemi
56 %
Sulamaya Kalan Süre
Toprak Nemi
TarasMobil Asistanı
Merhaba! Ben TarasMobil asistanınızım. Tarlanızdaki verileri analiz edebilir ve sorularınızı cevaplayabilirim.
Mesajınızı yazın...


21:37
11
TARAS
Kullanıcı Adı
Şifre
Giriş Yap
Hesabınız yok mu? Kayıt Ol
Yerel Demo
AWS Demo
English


21:33
TARAS
Ana Tarla
AWS
TA
Ayarlar
Tema Modu
Açık
Koyu
Sistem
Hava Durumu
Dil
Türkçe
English
AWS Bağlantı Testi
Çıkış Yap
Karbon
Çizelge
Ana Sayfa
Hastalık
Ayarlar

# 7. Conclusion

This document outlined the system architecture and methodology of the TARAS precision agriculture platform, focusing on its layered structure, modular subsystems and data-driven decision logic. By combining field sensing, agronomy-based irrigation analysis, AI-supported disease detection and user-centered interface design, TARAS provides a scalable and maintainable foundation for intelligent agricultural support. The proposed architecture enables future extensions while ensuring reliable and accessible system operation.

# 8. References

**[1]***Allen, R. G., Pereira, L. S., Raes, D., & Smith, M. (1998).*

*Crop evapotranspiration – Guidelines for computing crop water requirements.*

*FAO Irrigation and Drainage Paper No. 56.*

*Food and Agriculture Organization of the United Nations.*

https://www.fao.org/4/x0490e/x0490e00.htm

**[2]**Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016).
*Using deep learning for image-based plant disease detection.*
https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2016.01419/full
**[3]** Tan, M., & Le, Q. (2019).
*EfficientNetV2: Smaller Models and Faster Training*
Proceedings of the 36th International Conference on Machine Learning (ICML)
https://proceedings.mlr.press/v139/tan21a/tan21a.pdf

**[4]***Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014).*

*How transferable are features in deep neural networks?*

*Advances in Neural Information Processing Systems (NeurIPS).*

https://arxiv.org/abs/1411.1792

**[5]** Czaja, S. J., & Lee, C. C. (2007).

The impact of aging on access to technology.

Universal Access in the Information Society, 5(4), 341–349.

https://www.researchgate.net/publication/220606716_The_impact_of_aging_on_access_to_technologdr